FINANCIAL DATA ANALYST
BAYESIAN REGIME DETECTION ENGINE FOR EQUITY DIRECTION FORECASTING

Category: Bayesian Deep Learning | Regime Detection | Indian Equity Markets | Sequential Bayesian Inference & Model Ensembling

Timeline: 15 Days | AI-Accelerated Innovation & Research Project

TASK

Build a Bayesian Regime Detection Engine that classifies the Indian equity market into discrete states (risk-on, risk-off, transitional, late-cycle, post-shock) with calibrated uncertainty, integrating Hidden Markov Models, Bayesian Deep Learning, time-series foundation models, regime-switching vector autoregression, sequential Monte Carlo inference, and conformal prediction. The engine combines these sources through a documented ensembling layer to produce audit-defensible, time-varying regime probabilities with explicit uncertainty budgets - supporting real-time regime monitoring, allocation-tilt guidance, and regulator-grade probability reporting for long-only Indian mutual fund schemes.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:
This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, early career talent, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project positions early career talent as Data Quantitative Analysts on the Multi-Asset Solutions desk of an Asset Management Company (AMC), headquartered in Mumbai. The Chief Investment Officer has tasked the team with building a next-generation regime detection engine to inform tactical allocation tilts, cash buffer sizing, and sleeve weighting across the equity scheme line-up.

The core challenge: for three decades, quantitative inputs at Indian fund houses have orbited around price prediction - a problem the underlying data cannot honestly support over multi-month horizons. Signal-to-noise collapses, non-stationary regimes break model assumptions, and point forecasts cannot be defended to the Investment Committee or SEBI. Concurrently, structural flow dynamics in Indian equities have shifted: domestic institutional investors driven by monthly SIPs have become the dominant marginal buyer, alpha is compressing, and SEBI's regulatory direction increasingly favours documented, evidence-based, audit-defensible investment processes.

Early career talent will design Regime Lab - a gamified simulation platform that trains buy-side analysts through realistic Indian market regime detection scenarios. The platform combines Hidden Markov Models, Bayesian neural networks, time-series foundation models (Chronos, Moirai, TimesFM), a multivariate regime-switching VAR, and sequential Monte Carlo inference, all fused through a documented model-ensembling layer and wrapped in conformal prediction for finite-sample-valid coverage. The emphasis throughout is on depth of model building - multiple complementary regime models, combined and calibrated - rather than on reporting surfaces.

India's mutual fund industry crossed ₹70 lakh crore in AUM in 2025, with equity-oriented schemes accounting for over 60% of net inflows. Monthly SIP inflows have crossed ₹26,000 crore. This project bridges the gap between academic regime-detection theory and the production-grade, regulator-compatible quantitative workflows now required by Tier 1 Indian fund houses.

> *Direction over price. Calibrated probability over point forecast. A documented ensemble of complementary models over a single black box.*

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: DOMAIN FUNDAMENTALS - INDIAN MUTUAL FUND ECOSYSTEM AND THE DIRECTIONAL THESIS

A1.1 The Indian Mutual Fund Industry

The Indian mutual fund industry has undergone a structural transformation in the last decade. Assets under management have grown from approximately ₹12 lakh crore in 2015 to over ₹70 lakh crore in 2025, with equity-oriented schemes accounting for the largest share. The industry is supervised by the Securities and Exchange Board of India (SEBI), with the Association of Mutual Funds in India (AMFI) acting as the industry body.

Key structural characteristics:

- Domestic Institutional Investors (DIIs) - primarily mutual funds and insurance companies - have become the dominant marginal buyer of Indian equities, displacing Foreign Portfolio Investors (FPIs) as the primary price-setting force
- Monthly Systematic Investment Plan (SIP) inflows crossed ₹26,000 crore in 2025, representing a structural and largely price-insensitive flow into equity schemes
- SEBI's categorisation framework restricts scheme construction to defined investment universes (large-cap, mid-cap, small-cap, flexi-cap, multi-cap, focused, ELSS, sectoral/thematic)
- The Risk-O-Meter regime requires monthly risk classification disclosure based on a defined methodology
- Total Expense Ratio (TER) compression has reduced active fund economics, increasing pressure for differentiated investment process to justify active fees

Active vs. passive dynamics:

Empirical evidence from SPIVA India scorecards shows that an increasing share of actively-managed large-cap funds underperform their benchmarks net of fees over rolling five-year periods. Passive AUM (index funds and ETFs) has grown from a negligible base to over ₹10 lakh crore. This alpha-compression dynamic creates an existential need for active managers to demonstrate genuine, defensible information advantage - precisely the gap a calibrated, probabilistic regime engine fills.

A1.2 The Directional Thesis - Why Price Prediction Fails at MF Horizons

The Direction Over Price thesis identifies three structural problems with price-prediction approaches at long-only mutual fund horizons:

Signal-to-noise problem:

The variance of equity index returns over one-year horizons is dominated empirically by unpredictable shocks. The predictable component - using even the best macro, microstructure,

and behavioural features - accounts for at most a few percent of total variance. A point forecast of next-year Nifty levels is asking the data to do work the data cannot do. In-sample fits succeed precisely for the reason they fail out-of-sample: the model flexibility that captures noise prevents generalisation.

Non-stationarity problem:

Markets are not stationary; the relationship between features and forward returns changes across regimes. A model fit across multiple regimes implicitly averages across them, calibrated to no regime in particular. A model fit on a single regime is, by construction, fragile when the regime ends. Indian examples: value-factor strategies during the 2017-2020 growth cycle, momentum strategies during the 2018-2019 mid-cap drawdown, low-volatility strategies during the 2020 COVID liquidity shock.

Honest-uncertainty problem:

A point forecast ("the Nifty will rise 12% this year") is statistically untestable in real time and rhetorically dangerous. When the forecast misses - as it usually will - there is no principled way to distinguish between a forecast that was wrong because the world surprised the model and a forecast that was wrong because the model was always miscalibrated. Both look identical from outside, and both erode institutional credibility at the same rate.

> *The right output of a long-horizon model is a probability over direction, with an explicit and auditable uncertainty budget.*

A1.3 The Directional Reframe

The intellectual move from price to direction is small in form but large in consequence. It replaces a single unanswerable question with three more answerable ones: What regime is the market in? Which directions are gaining probability mass against others? How do joint dynamics of capital flows and macro variables shift the conditional probability of upward, sideways, or downward moves?

Each is more answerable for the same reason: it asks the data to discriminate, not extrapolate. Discrimination has a clear loss function, a tractable error structure, and the property that more data reliably improves the estimate. Extrapolation does not.

Direction is more useful at the level of the actual portfolio decision. A scheme manager does not need a Nifty target to decide whether to tilt toward value, raise cash, rotate from financials to industrials, or extend equity beta. She needs a directional read, conditioned on the present portfolio and a credible probability distribution over outcomes. Direction maps cleanly to portfolio actions; price targets, in our experience, almost never do.

A1.4 Regime Definitions for Indian Equity Markets

For this project, the regime taxonomy used is a five-state classification specifically calibrated to Indian equity market dynamics:

Regime	Description	Typical Indicators
Risk-On	Sustained equity rally, broad participation, expanding earnings revisions	Nifty 50-DMA > 200-DMA, India VIX < 14, FII inflows positive, breadth > 70% above 50-DMA
Risk-Off	Drawdown, deteriorating macro, deleveraging, flight to quality	Nifty < 200-DMA, India VIX > 22, FII outflows persistent, INR depreciation, gilt yields rising
Transitional	Regime boundary; conflicting indicators, regime probability spread	Mixed breadth, VIX 14-22, conflicting macro indicators, low directional conviction
Late-Cycle	Expansion mature, narrow leadership, valuation stretched, signs of fatigue	Nifty 50-DMA > 200-DMA but breadth narrow, valuation z-score > 1.5, sector rotation
Post-Shock	Stabilisation following acute drawdown; mean-reversion potential	Volatility decay, oversold breadth, credit spreads compressing, base-building

A1.5 SEBI Regulatory Context

SEBI's evolution over the last decade has been consistent in one direction: toward documented, defensible, evidence-based investment processes. Key regulatory artefacts that shape the design constraints of a regime detection engine:

- Scheme categorisation (Oct 2017) restricts investment universe; regime calls must respect category constraints
- Risk-O-Meter (Jan 2021) requires monthly risk classification using a defined six-point scale; a regime engine should feed into Risk-O-Meter inputs
- Stewardship Code (Mar 2020) requires AMCs to document active ownership decisions
- Stress testing for mid-cap and small-cap schemes (Feb 2024) requires documented liquidity-stress assumptions - regime states are natural conditioning variables for such tests
- SEBI's regulatory direction on AI/ML disclosure (consultations ongoing) signals expectation of explainable, auditable model outputs

Any quantitative engine deployed at a Tier 1 Indian AMC must therefore produce outputs that are: (a) calibrated against historical truth; (b) explainable at the feature and regime level; (c) documented with full lineage; and (d) usable as inputs into existing regulatory artefacts.

SECTION A2: BAYESIAN STATISTICAL FOUNDATIONS

A2.1 The Bayesian Framework

Bayesian inference treats unknown quantities as random variables with probability distributions. Given prior beliefs $P(\theta)$ over parameters θ and a likelihood $P(D|\theta)$ of observing data D under those parameters, Bayes' theorem combines them into a posterior:

$$P(\theta \mid D) = P(D \mid \theta) \cdot P(\theta) / P(D)$$

$P(\theta)$ = prior distribution (beliefs before seeing data)
$P(D \mid \theta)$ = likelihood (data-generating process)
$P(D)$ = marginal likelihood (normalising constant)
$P(\theta \mid D)$ = posterior (beliefs after data)

For regime detection, θ represents regime parameters (transition probabilities, regime-conditional return means and volatilities, observation-model weights) and D is the observed sequence of market features. The posterior $P(\theta \mid D)$ captures everything the data tells us about the regime structure, including its uncertainty - exactly what an Investment Committee needs.

A2.2 Why Bayesian for Regime Detection

- Calibrated uncertainty: Bayesian posteriors yield credible intervals, not just point estimates - directly aligning with the directional thesis's audit requirement
- Prior incorporation: Domain knowledge (regime persistence, typical regime durations, regime asymmetry between bull/bear) enters cleanly as priors
- Coherent updating: As new data arrives, posteriors update without re-fitting from scratch; this is operationally important for live deployment
- Natural handling of latent states: Regimes are unobserved; Bayesian inference is the principled framework for latent-variable estimation
- Hierarchical structure: Multi-pair, multi-sector, multi-scheme structure maps naturally to hierarchical Bayesian models

A2.3 Conjugate Priors and Closed-Form Updates

For certain prior-likelihood combinations, the posterior has the same parametric form as the prior, enabling exact closed-form updates. Key conjugate pairs relevant to regime modelling:

Likelihood	Conjugate Prior	Regime Application
Bernoulli	Beta(α, β)	Probability of regime persistence at next step
Categorical (regime label)	Dirichlet(α_1, ..., α_K)	Transition probability vector for K-regime HMM
Normal (known variance)	Normal(μ_0, σ_0^2)	Regime-conditional return mean updates
Normal (unknown σ^2)	Normal-Inverse-Gamma	Joint posterior over regime return mean and volatility

Likelihood	Conjugate Prior	Regime Application
Multivariate Normal	Normal-Inverse-Wishart	Regime-conditional covariance matrix over factor returns

A2.4 Python Implementation - Beta-Bernoulli Regime Persistence

```
import numpy as np
from scipy import stats

class RegimePersistencePosterior:
    """Beta-Bernoulli model for regime self-transition probability."""
    def __init__(self, alpha=2.0, beta=1.0):
        self.alpha = alpha  # prior pseudo-counts of persistence
        self.beta  = beta   # prior pseudo-counts of transition
    def update(self, persisted: int, transitioned: int):
        self.alpha += persisted
        self.beta  += transitioned
    def posterior_mean(self):
        return self.alpha / (self.alpha + self.beta)
    def credible_interval(self, level=0.95):
        lo = (1 - level) / 2
        hi = 1 - lo
        return (stats.beta.ppf(lo, self.alpha, self.beta),
                stats.beta.ppf(hi, self.alpha, self.beta))

# Example: 60 days of risk-on regime, 55 stayed, 5 transitioned
post = RegimePersistencePosterior()
post.update(persisted=55, transitioned=5)
print(f"P(stay in risk-on) = {post.posterior_mean():.3f}")
print(f"95% CI = {post.credible_interval()}")
```

A2.5 Markov Chain Monte Carlo (MCMC)

When posteriors are not analytically tractable - which is the typical case for any realistic regime model - we approximate them by sampling. Markov Chain Monte Carlo methods construct a Markov chain whose stationary distribution is the target posterior. Three workhorse algorithms:

Metropolis-Hastings:

Propose a candidate from a tractable proposal distribution; accept or reject based on the ratio of target densities. Conceptually simple but requires careful proposal tuning.

Gibbs sampling:

Sample each parameter from its full conditional distribution, holding others fixed. Particularly well-suited to hierarchical models where full conditionals are conjugate.

Hamiltonian Monte Carlo (HMC) / NUTS:

Uses gradient information to propose efficient moves through the posterior; the No-U-Turn Sampler (NUTS) adaptively tunes the trajectory length. Implemented in PyMC, Stan, and NumPyro. The default choice for modern Bayesian regime models.

A2.6 Variational Inference (VI)

Where MCMC is exact but slow, Variational Inference is approximate but fast. VI posits a tractable family of distributions $q(\theta; \varphi)$ and finds the member closest in KL-divergence to the true posterior:

```
φ* = argmin KL(q(θ; φ) || P(θ | D))
   = argmax ELBO(φ)
ELBO(φ) = E_q[log P(D, θ)] - E_q[log q(θ; φ)]
```

VI scales to large datasets and integrates naturally with deep learning via stochastic gradient methods (SVI, BBVI, ADVI). The trade-off: VI typically underestimates posterior variance (a known pathology of mean-field approximation), so calibration must be checked empirically - and conformal prediction wrappers can correct any residual miscalibration (covered in Section A6).

SECTION A3: HIDDEN MARKOV MODELS AND REGIME SWITCHING

A3.1 The HMM Framework

A Hidden Markov Model assumes that the observed sequence (market features over time) is generated by an unobserved Markov chain of regime states. Formally, an HMM is specified by:

- A set of K hidden states $S = \{s_1, ..., s_K\}$ (the regimes)
- A K×K transition probability matrix A where $A_{ij} = P(s_j \text{ at } t+1 \mid s_i \text{ at } t)$
- An emission distribution $P(o_t \mid s_t)$ - typically Gaussian for return observations
- An initial state distribution π over K states

Three canonical HMM problems and their algorithms:

Problem	Question	Algorithm
Evaluation	P(observations | model)	Forward algorithm - $O(K^2T)$
Decoding	Most likely state sequence	Viterbi algorithm - $O(K^2T)$
Learning	Parameter estimation	Baum-Welch (EM) - iterative

A3.2 Gaussian Mixture HMM for Regime Detection

The simplest production-grade regime model. Each regime is associated with a Gaussian distribution over returns (or a vector of features), and the transition matrix encodes regime persistence. The hmmlearn library provides a clean Python implementation.

```
import numpy as np
```

```
import pandas as pd
from hmmlearn import hmm

def fit_regime_hmm(returns: pd.Series, n_states=5, n_iter=200, seed=42):
    """Fit a Gaussian-emission HMM to a returns series."""
    X = returns.values.reshape(-1, 1)
    model = hmm.GaussianHMM(
        n_components=n_states,
        covariance_type="diag",
        n_iter=n_iter,
        random_state=seed,
        tol=1e-5
    )
    model.fit(X)
    states = model.predict(X)
    state_probs = model.predict_proba(X)
    return model, states, state_probs

# Example usage with Nifty daily returns
nifty = pd.read_csv('nifty_daily.csv', parse_dates=['Date'], index_col='Date')
nifty['Return'] = nifty['Close'].pct_change()
model, states, probs = fit_regime_hmm(nifty['Return'].dropna(), n_states=5)
```

A3.3 Regime Labelling and Identification

Vanilla HMMs are agnostic about which numeric state corresponds to which economic regime. Standard practice is to post-hoc label states by their fitted mean and volatility:

```
def label_regimes(model, K=5):
    """Map numeric states to economic regime names by mean/vol signature."""
    summary = []
    for i in range(K):
        mu = model.means_[i, 0]
        sig = np.sqrt(model.covars_[i, 0, 0])
        summary.append((i, mu, sig))
    # Sort by (mean, -vol): risk-on = high mean low vol; risk-off = low mean high vol
    summary.sort(key=lambda x: (x[1], -x[2]), reverse=True)
    labels = ['Risk-On', 'Late-Cycle', 'Transitional', 'Post-Shock', 'Risk-Off']
    mapping = {summary[r][0]: labels[r] for r in range(K)}
    return mapping
```

A3.4 Bayesian HMM via PyMC

The frequentist HMM gives point estimates of transition probabilities and emission parameters. The Bayesian HMM places priors on these quantities, samples the full posterior via MCMC, and yields credible intervals on every quantity of interest - including the regime probability at each time step.

```
import pymc as pm
import pytensor.tensor as pt

def build_bayesian_hmm(returns, K=5):
    """Bayesian Gaussian-emission HMM with Dirichlet transition rows."""
    T = len(returns)
    with pm.Model() as model:
        alpha_diag = 8.0
        alpha_off  = 1.0
        alpha_mat = np.eye(K) * alpha_diag + (1 - np.eye(K)) * alpha_off
        P  = pm.Dirichlet('P', a=alpha_mat, shape=(K, K))
        pi = pm.Dirichlet('pi', a=np.ones(K), shape=K)
        mu    = pm.Normal('mu', mu=0, sigma=0.02, shape=K)
        sigma = pm.HalfNormal('sigma', sigma=0.03, shape=K)
        states = pm.Categorical('states', p=pi, shape=T)
        obs = pm.Normal('obs', mu=mu[states], sigma=sigma[states], observed=returns)
        trace = pm.sample(2000, tune=1000, target_accept=0.95,
                          random_seed=42, chains=4)
    return model, trace
```

The Bayesian formulation is significantly more expensive to fit (minutes to hours rather than seconds) but yields the artefact a Tier 1 fund house actually needs: full posterior distributions over regime parameters and time-varying regime probabilities with credible intervals.

A3.5 Regime-Switching Models - Pointer

Single-feature HMMs are the entry point. The full multivariate treatment - where the regime conditions the joint dynamics of returns, volatility, breadth, FII flows, and macro variables through a Regime-Switching Vector Autoregression - is developed in its own dedicated section (Section A8), reflecting its central role in the expanded model stack for this project.

A3.6 R Implementation

```
library(depmixS4)   # Hidden Markov Models in R
library(MSwM)       # Markov-switching regression

# depmixS4 example: 5-regime Gaussian HMM on Nifty returns
returns <- diff(log(nifty_close))
mod <- depmix(returns ~ 1, family = gaussian(),
              nstates = 5, data = data.frame(returns))
fit  <- fit(mod)
post <- posterior(fit)   # state and posterior probabilities

# MSwM example: regime-switching mean-variance model
mod_lm <- lm(returns ~ 1)
msm <- msmFit(mod_lm, k = 3, sw = c(TRUE, TRUE))
summary(msm)
```

A3.7 Evaluating Regime Models

Regime models cannot be evaluated against ground truth (regimes are latent). Practical evaluation criteria:

- In-sample log-likelihood and AIC/BIC for parsimony
- Out-of-sample one-step-ahead predictive likelihood
- Stability of regime durations against historical narratives
- Coherence with macro indicators (do risk-off regimes coincide with rate-cut announcements, FII outflows, INR depreciation?)
- Calibration of forward return probabilities (conformal coverage)
- Stability of regime assignment under rolling re-fits

SECTION A4: BAYESIAN DEEP LEARNING FOR REGIME DETECTION

A4.1 Motivation

Classical HMMs assume Gaussian emissions and linear dynamics. Real markets exhibit fat tails, non-linear feature interactions, and rich high-dimensional structure (sector breadth, options-market implied moments, derivatives positioning, alt-data). Bayesian neural networks let us absorb this complexity while preserving calibrated uncertainty quantification.

Three production-ready Bayesian deep-learning approaches for regime probability estimation:

- Monte Carlo Dropout (Gal & Ghahramani, 2016) - approximate Bayesian inference via repeated stochastic forward passes with dropout active
- Variational Bayesian Neural Networks - replace deterministic weights with variational distributions, optimised via stochastic gradient VI
- Deep Ensembles - train multiple networks with different initialisations; the predictive distribution is the mixture across ensemble members

A4.2 Monte Carlo Dropout

```
import tensorflow as tf
from tensorflow.keras import layers, Model

def build_mc_dropout_regime_classifier(input_dim, n_regimes=5, dropout_rate=0.3):
    """Regime classifier with permanent dropout for MC inference."""
    inputs = layers.Input(shape=(input_dim,))
    x = layers.Dense(128, activation='relu')(inputs)
    x = layers.Dropout(dropout_rate)(x, training=True)  # always on
    x = layers.Dense(64, activation='relu')(x)
    x = layers.Dropout(dropout_rate)(x, training=True)
    x = layers.Dense(32, activation='relu')(x)
    x = layers.Dropout(dropout_rate)(x, training=True)
    outputs = layers.Dense(n_regimes, activation='softmax')(x)
```

```
    model = Model(inputs, outputs)
    model.compile(optimizer='adam', loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

def mc_predict(model, X, n_samples=200):
    """Generate predictive distribution via repeated stochastic passes."""
    preds = np.stack([model(X, training=True).numpy() for _ in range(n_samples)])
    mean = preds.mean(axis=0)
    std  = preds.std(axis=0)
    return mean, std, preds  # preds shape: (n_samples, batch, n_regimes)
```

The standard deviation of the predictive distribution at each time step gives the model's uncertainty about the regime call. High mean probability paired with low standard deviation = high-conviction regime classification; high standard deviation = transitional or out-of-distribution market state.

A4.3 Variational Bayesian Neural Network

```
import tensorflow_probability as tfp
tfd = tfp.distributions
tfpl = tfp.layers

def build_variational_regime_classifier(input_dim, n_regimes=5, train_size=2000):
    """Mean-field VI BNN for regime classification."""
    kl_weight = 1.0 / train_size
    inputs = tf.keras.Input(shape=(input_dim,))
    x = tfpl.DenseFlipout(128, activation='relu',
            kernel_divergence_fn=lambda q,p,_: kl_weight*tfd.kl_divergence(q,p))(inputs)
    x = tfpl.DenseFlipout(64, activation='relu',
            kernel_divergence_fn=lambda q,p,_: kl_weight*tfd.kl_divergence(q,p))(x)
    logits = tfpl.DenseFlipout(n_regimes,
            kernel_divergence_fn=lambda q,p,_: kl_weight*tfd.kl_divergence(q,p))(x)
    outputs = tfpl.OneHotCategorical(n_regimes)(logits)
    model = tf.keras.Model(inputs, outputs)
    nll = lambda y, rv_y: -rv_y.log_prob(y)
    model.compile(optimizer='adam', loss=nll, metrics=['accuracy'])
    return model
```

A4.4 Deep Ensembles

Deep ensembles are conceptually the simplest yet empirically among the most reliable approaches to Bayesian deep learning. Train M independent networks with different random initialisations; the predictive distribution is the mixture over ensemble members:

```
def train_deep_ensemble(build_fn, X_train, y_train, M=10, epochs=50):
    """Train M independent regime classifiers."""
```

```
    ensemble = []
    for m in range(M):
        tf.random.set_seed(m * 17 + 3)
        model = build_fn()
        model.fit(X_train, y_train, epochs=epochs, batch_size=64, verbose=0)
        ensemble.append(model)
    return ensemble

def ensemble_predict(ensemble, X):
    preds = np.stack([m.predict(X, verbose=0) for m in ensemble])
    mean = preds.mean(axis=0)
    epistemic = preds.std(axis=0)           # disagreement across members
    aleatoric = (preds * (1 - preds)).mean(axis=0)  # per-member uncertainty
    return mean, epistemic, aleatoric
```

Critically, deep ensembles separate epistemic uncertainty (model uncertainty - how much do ensemble members disagree?) from aleatoric uncertainty (irreducible noise in the data-generating process). For an Investment Committee, this distinction matters: epistemic uncertainty can in principle be reduced with more data; aleatoric cannot. Communicating which kind of uncertainty dominates a regime call sharpens the investment decision considerably.

A4.5 Feature Engineering for Regime Classification

```
def engineer_regime_features(df: pd.DataFrame) -> pd.DataFrame:
    """Build feature matrix for Indian equity regime classification."""
    f = pd.DataFrame(index=df.index)
    # Return features
    f['ret_1d']  = df['Close'].pct_change()
    f['ret_5d']  = df['Close'].pct_change(5)
    f['ret_21d'] = df['Close'].pct_change(21)
    f['ret_63d'] = df['Close'].pct_change(63)
    # Trend features
    f['ma_50_200'] = (df['Close'].rolling(50).mean() / df['Close'].rolling(200).mean() - 1)
    f['above_200dma'] = (df['Close'] > df['Close'].rolling(200).mean()).astype(int)
    # Volatility features
    f['vol_21d'] = df['Close'].pct_change().rolling(21).std() * np.sqrt(252)
    f['vol_63d'] = df['Close'].pct_change().rolling(63).std() * np.sqrt(252)
    f['vol_ratio'] = f['vol_21d'] / f['vol_63d']
    # India VIX features
    f['vix_level'] = df['IndiaVIX']
    f['vix_change_5d'] = df['IndiaVIX'].pct_change(5)
    f['vix_z']      =      (df['IndiaVIX']      -      df['IndiaVIX'].rolling(252).mean())      /
df['IndiaVIX'].rolling(252).std()
    # Breadth features
    f['adv_dec_ratio'] = df['Advances'] / df['Declines']
    f['pct_above_50dma'] = df['PctAbove50DMA']
```

```
    f['new_highs_lows'] = df['NewHighs'] - df['NewLows']
    # Macro features
    f['gilt_10y_change_21d'] = df['Gilt10Y'].diff(21)
    f['inr_change_21d'] = df['USDINR'].pct_change(21)
    f['credit_spread'] = df['AAA10Y'] - df['Gilt10Y']
    # Flow features
    f['fii_eq_5d'] = df['FII_Equity'].rolling(5).sum()
    f['dii_eq_5d'] = df['DII_Equity'].rolling(5).sum()
    f['flow_balance'] = f['dii_eq_5d'] / (abs(f['fii_eq_5d']) + 1e-9)
    return f.dropna()
```

A4.6 Training Pipeline with Time-Aware Cross-Validation

```
from sklearn.model_selection import TimeSeriesSplit

def regime_label_from_hmm(returns, n_states=5):
    """Generate pseudo-ground-truth regime labels from HMM for supervised training."""
    model, states, _ = fit_regime_hmm(returns, n_states=n_states)
    mapping = label_regimes(model)
    return np.array([mapping[s] for s in states]), model

def train_bayesian_regime_pipeline(X, y, build_fn, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    fold_results = []
    for fold, (train_idx, test_idx) in enumerate(tscv.split(X)):
        X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
        y_tr, y_te = y[train_idx], y[test_idx]
        ensemble = train_deep_ensemble(build_fn, X_tr, y_tr, M=8)
        mean, epi, ale = ensemble_predict(ensemble, X_te)
        fold_results.append({
            'fold': fold, 'mean_acc': (mean.argmax(1) == y_te).mean(),
            'epi_mean': epi.mean(), 'ale_mean': ale.mean()
        })
    return fold_results
```

SECTION A5: FOUNDATION MODELS FOR TIME-SERIES

A5.1 Why Foundation Models Now

The last two years have seen the emergence of pre-trained foundation models for time-series - Chronos (Amazon), Moirai (Salesforce), TimesFM (Google), Lag-Llama (ServiceNow). These models, trained on tens of thousands of time-series across domains, encode general structural priors about temporal data: trend, seasonality, regime changes, intermittency, scale invariance. Their effect on financial modelling is analogous to the effect transformer-based language models had on NLP: hand-engineered features are replaced by pre-trained embeddings that already encode temporal structure.

For regime detection, foundation models contribute in three ways:

- Sample efficiency - pre-training amortises the cost of learning generic temporal patterns; downstream regime classification needs far fewer Indian-specific examples
- Transfer across markets - a foundation model trained on global equities transfers reasonably to Indian equities, then can be fine-tuned
- Embedding-based features - even without forecasting, the latent representations from a foundation model serve as rich input features to a downstream classifier

In the expanded model stack for this project, at least two distinct foundation models are mandatory (not optional), so that transfer behaviour and embedding quality can be compared head-to-head on the Indian regime task rather than assumed.

A5.2 Chronos for Zero-Shot Forecasting and Embedding Extraction

Chronos tokenises time-series values into discrete bins and applies a language-model architecture. It is available in several sizes (Tiny, Small, Base, Large) and can be invoked zero-shot or fine-tuned. For regime detection, we use it as an embedding extractor:

```
from chronos import ChronosPipeline
import torch

pipeline = ChronosPipeline.from_pretrained(
    "amazon/chronos-t5-base",
    device_map="cuda",
    torch_dtype=torch.bfloat16,
)

def chronos_embed(returns_window: np.ndarray):
    """Extract Chronos embedding for a window of returns."""
    context = torch.tensor(returns_window, dtype=torch.bfloat16)
    embeddings, tokenizer_state = pipeline.embed(context)
    return embeddings.mean(dim=1).cpu().numpy()  # pool to fixed vector

# Generate embeddings for rolling 252-day windows
window = 252
embeddings = []
for i in range(window, len(nifty_returns)):
    emb = chronos_embed(nifty_returns.iloc[i-window:i].values)
    embeddings.append(emb)
embeddings = np.vstack(embeddings)  # (T-window, d_model)
# Feed these embeddings into the Bayesian classifier from A4
```

A5.3 TimesFM, Lag-Llama and Moirai

TimesFM (Google Research) uses a decoder-only transformer trained on 100B time-points spanning a diverse mix of synthetic and real-world series. Lag-Llama uses a similar architecture

but emphasises probabilistic forecasting via student-t output distributions - a natural fit for the fat-tailed return distributions of Indian equities. Moirai (Salesforce) adds any-variate masked attention, allowing multiple market features to be encoded jointly. The project requires comparing at least two of these against Chronos on sample-efficiency and embedding-quality curves.

```
# TimesFM example (lightweight usage)
import timesfm
tfm = timesfm.TimesFm(
   context_len=512, horizon_len=128, input_patch_len=32,
   output_patch_len=128, num_layers=20, model_dims=1280, backend='gpu',
)
tfm.load_from_checkpoint(repo_id="google/timesfm-1.0-200m")

point_forecast, experimental_quantile_forecast = tfm.forecast(
   [nifty_returns.values], freq=[0]   # 0=high freq, 1=medium, 2=low
)
# Use quantiles as features for downstream regime classification
```

A5.4 Hybrid Architecture - Foundation Embedding + Bayesian Head

The architecture we recommend for production deployment at a Tier 1 Indian AMC is hybrid: foundation-model embeddings on the input side, Bayesian classification head on the output side. The foundation model captures rich temporal structure; the Bayesian head delivers calibrated probabilities with full uncertainty quantification.

```
class HybridRegimeModel:
   def __init__(self, foundation_model, bayesian_classifier):
      self.fm  = foundation_model
      self.clf = bayesian_classifier  # MC dropout / ensemble / VI
   def encode(self, X_windows):
      return np.vstack([self.fm.embed(w) for w in X_windows])
   def fit(self, X_windows, y, epochs=50):
      Z = self.encode(X_windows)
      self.clf.fit(Z, y, epochs=epochs)
   def predict_with_uncertainty(self, X_windows, n_mc=200):
      Z = self.encode(X_windows)
      preds = np.stack([self.clf(Z, training=True).numpy() for _ in range(n_mc)])
      mean = preds.mean(0); std = preds.std(0)
      return mean, std
```

SECTION A6: CONFORMAL PREDICTION AND CALIBRATION

A6.1 The Calibration Problem

A model that reports “75% probability of risk-on regime over the next month” is useful only if, empirically, 75% of such forecasts actually realise as risk-on outcomes. This property -

calibration - is famously violated by deep neural networks (Guo et al., 2017) and partially violated by variational Bayesian neural networks. For a fiduciary investment process, miscalibration is not a statistical curiosity; it is an existential risk to institutional credibility.

Conformal prediction, developed primarily by Vladimir Vovk and collaborators since the 1990s, provides a model-agnostic framework for producing prediction sets with finite-sample-valid coverage guarantees, independent of the underlying model's distributional assumptions or correctness.

A6.2 Split-Conformal Prediction

Split-conformal prediction works as follows for a classification task with K classes:

- Split data into proper training set and calibration set
- Train any base model on the training set
- For each calibration example, compute a non-conformity score (e.g., 1 - probability assigned to true class)
- For desired coverage level 1-α, find the (1-α)-quantile $\hat{q}$ of the calibration scores
- For new examples, the prediction set is {classes with score below $\hat{q}$}

Marginal coverage guarantee: for exchangeable data, the prediction set contains the true label with probability ≥ 1-α.

```
def split_conformal_classifier(model, X_cal, y_cal, X_test, alpha=0.1):
    """Return prediction sets with marginal coverage 1-alpha."""
    cal_probs = model.predict(X_cal)
    cal_scores = 1 - cal_probs[np.arange(len(y_cal)), y_cal]
    n = len(cal_scores)
    q_level = np.ceil((n + 1) * (1 - alpha)) / n
    q_hat = np.quantile(cal_scores, q_level, interpolation='higher')
    test_probs = model.predict(X_test)
    pred_sets = test_probs >= (1 - q_hat)   # boolean mask of regimes in set
    return pred_sets, q_hat
```

A6.3 Adaptive Prediction Sets (APS)

Standard split-conformal classification can produce trivial prediction sets (all classes or no classes) when the underlying model is poorly calibrated. Adaptive Prediction Sets (Romano et al., 2020) compute the non-conformity score as the cumulative probability of classes sorted from most to least probable, producing more efficient sets:

```
def adaptive_prediction_sets(model, X_cal, y_cal, X_test, alpha=0.1):
    cal_probs = model.predict(X_cal)
    sorted_idx = np.argsort(-cal_probs, axis=1)
    sorted_probs = np.take_along_axis(cal_probs, sorted_idx, axis=1)
    cumsum = np.cumsum(sorted_probs, axis=1)
```

```
    rank_of_true = np.array([
        np.where(sorted_idx[i] == y_cal[i])[0][0] for i in range(len(y_cal))])
    cal_scores = cumsum[np.arange(len(y_cal)), rank_of_true]
    n = len(cal_scores)
    q_level = np.ceil((n + 1) * (1 - alpha)) / n
    q_hat = np.quantile(cal_scores, q_level, interpolation='higher')
    test_probs = model.predict(X_test)
    sorted_idx_t = np.argsort(-test_probs, axis=1)
    sorted_probs_t = np.take_along_axis(test_probs, sorted_idx_t, axis=1)
    cumsum_t = np.cumsum(sorted_probs_t, axis=1)
    in_set = cumsum_t <= q_hat
    in_set[:, 0] = True   # always include top class
    return in_set, sorted_idx_t
```

A6.4 Conformalised Quantile Regression for Return Forecasts

For continuous forward-return forecasts (rather than discrete regime labels), Conformalised Quantile Regression (CQR; Romano, Patterson, Candès 2019) produces prediction intervals with coverage guarantees. CQR uses a base quantile regressor and uses calibration data to correct any miscalibration.

```
from sklearn.ensemble import GradientBoostingRegressor

def cqr_intervals(X_tr, y_tr, X_cal, y_cal, X_te, alpha=0.1):
    """Conformalised quantile regression intervals."""
    lo, hi = alpha/2, 1 - alpha/2
    q_lo = GradientBoostingRegressor(loss='quantile', alpha=lo, n_estimators=200)
    q_hi = GradientBoostingRegressor(loss='quantile', alpha=hi, n_estimators=200)
    q_lo.fit(X_tr, y_tr); q_hi.fit(X_tr, y_tr)
    lo_cal, hi_cal = q_lo.predict(X_cal), q_hi.predict(X_cal)
    E = np.maximum(lo_cal - y_cal, y_cal - hi_cal)
    n = len(E)
    q = np.quantile(E, np.ceil((n+1)*(1-alpha))/n, interpolation='higher')
    lo_te, hi_te = q_lo.predict(X_te), q_hi.predict(X_te)
    return lo_te - q, hi_te + q
```

A6.5 Time-Series and Online Conformal Methods

The marginal coverage guarantee assumes exchangeability - an assumption financial time-series violate. The expanded model stack therefore requires the candidate to go beyond split-conformal and implement at least one distribution-shift-robust variant:

- EnbPI (Ensemble Batch Prediction Intervals; Xu & Xie, 2021) - leave-one-out ensemble residuals give prediction intervals without a held-out calibration set, well-suited to sequential data

- Adaptive Conformal Inference (ACI; Gibbs & Candès, 2021) - the miscoverage level α_t is updated online so realised coverage tracks the target even under regime shift
- Mondrian / class-conditional conformal - separate calibration per regime class, so coverage holds within each regime rather than only on average across regimes

```
def adaptive_conformal_inference(scores_stream, alpha_target=0.1, gamma=0.01):
    """Online ACI: update alpha_t after each realised outcome.
       scores_stream yields (nonconformity_score, q_hat, covered) per step."""
    alpha_t = alpha_target
    coverage_path = []
    for score, q_hat, covered in scores_stream:
        # err_t = 1 if the true label fell OUTSIDE the prediction set
        err_t = 0 if covered else 1
        alpha_t = alpha_t + gamma * (alpha_target - err_t)
        alpha_t = min(max(alpha_t, 1e-3), 1 - 1e-3)
        coverage_path.append((alpha_t, covered))
    return coverage_path
```

A6.6 Empirical Calibration Diagnostics

Even with conformal wrappers, ongoing monitoring of empirical coverage is mandatory for production deployment. The reliability diagram and the Expected Calibration Error (ECE) are standard tools:

```
def reliability_diagram(probs_max, correct, n_bins=10):
    bins = np.linspace(0, 1, n_bins + 1)
    bin_lowers, bin_uppers = bins[:-1], bins[1:]
    accuracies, confidences, counts = [], [], []
    for lo, hi in zip(bin_lowers, bin_uppers):
        in_bin = (probs_max > lo) & (probs_max <= hi)
        if in_bin.sum() > 0:
            accuracies.append(correct[in_bin].mean())
            confidences.append(probs_max[in_bin].mean())
            counts.append(in_bin.sum())
        else:
            accuracies.append(0); confidences.append((lo+hi)/2); counts.append(0)
    return np.array(accuracies), np.array(confidences), np.array(counts)

def expected_calibration_error(probs_max, correct, n_bins=10):
    acc, conf, cnt = reliability_diagram(probs_max, correct, n_bins)
    return np.sum(cnt / cnt.sum() * np.abs(acc - conf))
```

> *Calibration is not a statistical curiosity; it is the foundation on which an investment committee can trust a model output.*

SECTION A7: TOPOLOGICAL DATA ANALYSIS AND GRAPH STRUCTURE

A7.1 Why Topology

Markets often exhibit structural changes that are invisible to point-based metrics but visible in the shape of the multi-dimensional feature manifold. Topological Data Analysis (TDA), in particular persistent homology, gives a principled way to detect such changes by tracking how topological features (connected components, loops, voids) of the feature point cloud evolve as a similarity threshold sweeps from zero to infinity.

- Persistent homology of rolling correlation matrices identifies regime breaks before they appear in price-only metrics
- Persistence landscapes (Bubenik 2015) provide stable numeric summaries usable as classifier features
- The Mapper algorithm produces interpretable 2-D visualisations of the high-dimensional regime manifold

A7.2 Persistent Homology via giotto-tda

```
from gtda.homology import VietorisRipsPersistence
from gtda.diagrams import PersistenceLandscape

def topology_features(corr_matrices_window):
    """Extract persistence landscape features from rolling correlation matrices.
       corr_matrices_window: (T, N, N) tensor of correlation matrices."""
    distances = np.sqrt(2 * (1 - corr_matrices_window))   # d = sqrt(2*(1-rho))
    VR = VietorisRipsPersistence(metric='precomputed',
                                 homology_dimensions=[0, 1], n_jobs=-1)
    diagrams = VR.fit_transform(distances)
    PL = PersistenceLandscape(n_layers=5, n_bins=100)
    landscapes = PL.fit_transform(diagrams)
    return landscapes.reshape(landscapes.shape[0], -1)
```

A7.3 Graph Neural Networks for Sector Structure

Complementary to TDA, Graph Neural Networks (GNNs) treat the universe of stocks/sectors as a graph with edges weighted by similarity or co-movement. A temporal GNN can encode the evolving structure of sector relationships and pass this into the regime classifier as a sector-aware feature. The detailed temporal-graph architecture is the subject of Zetheta's second project concept; here we use a lightweight GCN as one input to the regime engine.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import GCNConv

class SectorGCN(nn.Module):
    def __init__(self, in_dim, hidden=64, out_dim=32):
```

```
        super().__init__()
        self.conv1 = GCNConv(in_dim, hidden)
        self.conv2 = GCNConv(hidden, out_dim)
    def forward(self, x, edge_index, edge_weight=None):
        x = F.relu(self.conv1(x, edge_index, edge_weight))
        x = self.conv2(x, edge_index, edge_weight)
        return x.mean(dim=0)   # graph-level embedding for the day
```

SECTION A8: REGIME-SWITCHING VECTOR AUTOREGRESSION AND MULTIVARIATE DYNAMICS

Single-feature HMMs treat the regime as conditioning only the mean and volatility of one return series. In reality, a regime conditions the joint dynamics of many series at once: returns, realised volatility, breadth, FII and DII flows, INR, and gilt yields all co-move differently in a risk-off regime than in a risk-on regime. The Regime-Switching Vector Autoregression (RS-VAR; Hamilton 1989) is the canonical model for this, and it is a core deliverable of the expanded model stack.

A8.1 The RS-VAR Specification

Each regime owns its own VAR coefficient matrix and innovation covariance, and the regime itself follows a latent Markov chain:

```
# Regime-switching VAR(1) structure
# y_t = c_{s_t} + A_{s_t} y_{t-1} + e_t,  e_t ~ N(0, Sigma_{s_t})
# s_t ~ Markov chain with transition matrix P
#
# y_t is a vector of features (Nifty return, vol, breadth, FII, DII, INR, gilt)
# Each regime s has its own intercept c_s, dynamics A_s, and covariance Sigma_s
```

A8.2 Single-Feature Markov-Switching Baseline (statsmodels)

The single-feature Markov-switching regression is the natural starting baseline before building the full multivariate Bayesian model:

```
import statsmodels.api as sm
from statsmodels.tsa.regime_switching.markov_regression import MarkovRegression

def fit_msm_regression(y, k_regimes=3, switching_variance=True):
    mod = MarkovRegression(y, k_regimes=k_regimes, trend='c',
                           switching_variance=switching_variance)
    res = mod.fit()
    return res

nifty_returns = pd.read_csv('nifty.csv')['Return'].dropna()
res = fit_msm_regression(nifty_returns.values, k_regimes=3)
print(res.summary())
```

```
print(res.smoothed_marginal_probabilities[2].tail(10))  # P(high-vol | data)
```

A8.3 Full Multivariate Bayesian RS-VAR (PyMC / NumPyro)

The production path is a custom Bayesian RS-VAR, because it yields full posterior coverage over regime-conditional dynamics and innovation covariances - the artefact a Tier 1 AMC needs for stress conditioning. The latent state path is marginalised with a forward-filter / backward-sample step; emissions are multivariate-normal per regime with an LKJ prior on each regime's correlation structure.

```
import pymc as pm
import pytensor.tensor as pt
import numpy as np

def build_bayesian_rsvar(Y, K=5):
    """Bayesian regime-switching VAR(1). Y: (T, d) feature matrix."""
    T, d = Y.shape
    with pm.Model() as model:
        # Persistence-favouring Dirichlet rows for the transition matrix
        alpha = np.eye(K) * 8.0 + (1 - np.eye(K)) * 1.0
        P  = pm.Dirichlet('P', a=alpha, shape=(K, K))
        pi = pm.Dirichlet('pi', a=np.ones(K), shape=K)
        # Regime-conditional intercepts and VAR(1) coefficient matrices
        c = pm.Normal('c', 0.0, 0.05, shape=(K, d))
        Avar = pm.Normal('A', 0.0, 0.3, shape=(K, d, d))
        # Regime-conditional covariance via LKJ + half-normal scales
        chol, corr, sd = pm.LKJCholeskyCov(
            'chol', n=d, eta=2.0,
            sd_dist=pm.HalfNormal.dist(0.03, shape=d), compute_corr=True)
        # Latent Markov chain over regimes (marginalised in inference)
        states = pm.HiddenMarkovChain('states', P=P, init_dist=pi, shape=T)
        mu = c[states] + pt.batched_dot(Avar[states], Y[:-1])
        pm.MvNormal('obs', mu=mu, chol=chol[states], observed=Y[1:])
        trace = pm.sample(1500, tune=1500, target_accept=0.95,
                          chains=4, random_seed=42)
    return model, trace
```

Note: where pm.HiddenMarkovChain is unavailable, the candidate marginalises the latent chain manually via a forward-algorithm log-likelihood term, or implements the model in NumPyro using a scan-based forward pass for speed. Both routes are acceptable; the binding requirement is full posterior coverage with documented MCMC diagnostics (R-hat < 1.01, sufficient ESS, zero divergences after tuning).

A8.4 Reading the RS-VAR Output

- Regime-conditional impulse responses: how a one-standard-deviation FII-outflow shock propagates to Nifty returns differs by regime - directly informative for stress conditioning

- Regime-conditional covariance: cross-asset correlations spike in risk-off regimes; the posterior over Sigma_s makes this explicit with credible intervals
- Smoothed regime probabilities: the time-varying P(regime | full history) path, with credible bands, feeds the ensembling layer in Section A10

SECTION A9: SEQUENTIAL AND ONLINE REGIME INFERENCE

Batch MCMC fits the whole history at once and is too slow to rerun every trading day. Production regime engines need sequential inference: update the regime posterior as each new day arrives, in seconds rather than hours. This section develops three complementary online methods - particle filtering, Bayesian online changepoint detection, and streaming posterior updates - which together form the live-inference layer of the engine.

A9.1 Sequential Monte Carlo (Particle Filter) for Regime State

A particle filter represents the regime posterior at each time t by a weighted set of particles. As each observation arrives, particles are propagated through the transition matrix, re-weighted by the emission likelihood, and resampled. This yields an exact-in-the-limit online estimate of P(regime_t | observations up to t).

```
import numpy as np

class RegimeParticleFilter:
    """Bootstrap particle filter for a K-regime HMM with Gaussian emissions."""
    def __init__(self, P, mu, sigma, n_particles=5000, seed=42):
        self.P = P            # (K, K) transition matrix
        self.mu = mu          # (K,) regime means
        self.sigma = sigma    # (K,) regime stds
        self.K = P.shape[0]
        self.N = n_particles
        self.rng = np.random.default_rng(seed)
        self.particles = self.rng.integers(0, self.K, size=self.N)
        self.weights = np.full(self.N, 1.0 / self.N)

    def step(self, obs):
        # 1) propagate each particle through the transition matrix
        self.particles = np.array([
            self.rng.choice(self.K, p=self.P[s]) for s in self.particles])
        # 2) re-weight by Gaussian emission likelihood
        like = np.exp(-0.5 * ((obs - self.mu[self.particles]) /
                              self.sigma[self.particles]) ** 2) / (self.sigma[self.particles] *
np.sqrt(2 * np.pi))
        self.weights *= like
        self.weights /= self.weights.sum()
        # 3) systematic resample if effective sample size is low
```

```
        ess = 1.0 / np.sum(self.weights ** 2)
        if ess < self.N / 2:
            idx = self.rng.choice(self.N, size=self.N, p=self.weights)
            self.particles = self.particles[idx]
            self.weights = np.full(self.N, 1.0 / self.N)
        # regime posterior at this step
        post = np.bincount(self.particles, weights=self.weights, minlength=self.K)
        return post / post.sum()
```

A9.2 Bayesian Online Changepoint Detection (BOCPD)

BOCPD (Adams & MacKay, 2007) maintains a posterior over the run length - the number of steps since the last regime break - and flags a changepoint when the run-length distribution collapses toward zero. It is the natural complement to the smooth regime models: BOCPD detects the discontinuity, the HMM/RS-VAR classifies the new state.

```
import numpy as np
from scipy.stats import norm

def bocpd(data, hazard=1/100, mu0=0.0, kappa0=1.0, alpha0=1.0, beta0=1.0):
    """Bayesian online changepoint detection with a Normal-Gamma model.
       Returns the run-length posterior matrix R (T+1, T+1)."""
    T = len(data)
    R = np.zeros((T + 1, T + 1)); R[0, 0] = 1.0
    mu, kappa, alpha, beta = [mu0], [kappa0], [alpha0], [beta0]
    for t, x in enumerate(data):
        # predictive prob of x under each run length (Student-t)
        scale = np.sqrt(np.array(beta) * (np.array(kappa) + 1) /
                        (np.array(alpha) * np.array(kappa)))
        pred = norm.pdf(x, loc=np.array(mu), scale=scale)
        R[1:t+2, t+1] = R[0:t+1, t] * pred * (1 - hazard)   # growth
        R[0, t+1] = np.sum(R[0:t+1, t] * pred * hazard)      # changepoint
        R[:, t+1] /= R[:, t+1].sum()
        # update sufficient statistics (Normal-Gamma conjugacy)
        mu_new   = (np.array(kappa) * np.array(mu) + x) / (np.array(kappa) + 1)
        kappa_new = np.array(kappa) + 1
        alpha_new = np.array(alpha) + 0.5
        beta_new  = np.array(beta) + (np.array(kappa) * (x - np.array(mu))**2)                    / (2 *
(np.array(kappa) + 1))
        mu    = np.concatenate([[mu0], mu_new])
        kappa = np.concatenate([[kappa0], kappa_new])
        alpha = np.concatenate([[alpha0], alpha_new])
        beta  = np.concatenate([[beta0], beta_new])
    return R
```

In the Indian context, BOCPD should fire sharply on the 2020 COVID break and the 2018 IL&FS credit shock, and should NOT fire on routine intraday noise - a property the candidate must demonstrate empirically against the case-study episodes in Part C.

A9.3 Streaming Posterior Updates and the Online/Batch Reconciliation

The Beta-Bernoulli and Dirichlet conjugacy of Section A2 means transition-probability posteriors update in closed form as new regime transitions are observed. The operational design is a two-speed engine: a nightly batch RS-VAR / Bayesian-HMM refit establishes the reference posterior, and an intraday particle filter plus BOCPD tracks the state between refits. A reconciliation check - does the fast online posterior agree with the last batch posterior on the overlap window? - is a mandatory model-health diagnostic.

SECTION A10: MODEL ENSEMBLING, STACKING, AND SELECTION

No single regime model dominates across all market conditions: HMMs are robust but rigid, Bayesian neural networks are flexible but data-hungry, foundation-model heads are sample-efficient but opaque, and the RS-VAR captures cross-asset structure the others ignore. The engine's final regime probability is therefore an explicit, documented combination of these sources - not a single black box. This is the integration layer that turns a collection of models into one audit-defensible output.

A10.1 Bayesian Model Averaging

Bayesian Model Averaging (BMA) weights each model by its marginal evidence (or a predictive-likelihood proxy), so the combined probability automatically tilts toward whichever model is currently explaining the data best:

```
import numpy as np

def bma_weights(log_predictive_likelihoods):
    """log_predictive_likelihoods: (M,) out-of-sample log-lik per model."""
    w = np.exp(log_predictive_likelihoods - log_predictive_likelihoods.max())
    return w / w.sum()

def bma_combine(model_probs, weights):
    """model_probs: (M, K) regime probs per model; weights: (M,)."""
    return np.tensordot(weights, model_probs, axes=(0, 0))  # (K,)
```

A10.2 Stacking with a Time-Aware Meta-Learner

Stacking learns the combination weights from data rather than fixing them. A constrained meta-learner (weights non-negative, summing to one) is trained on out-of-fold base-model probabilities using a strictly time-respecting split, so no future information leaks into the combination:

```
from sklearn.model_selection import TimeSeriesSplit
```

```
from scipy.optimize import minimize

def fit_stacking_weights(base_probs, y_true):
    """base_probs: (M, N, K) out-of-fold probs; y_true: (N,) labels.
       Returns simplex weights minimising mean cross-entropy."""
    M, N, K = base_probs.shape
    onehot = np.eye(K)[y_true]
    def neg_loglik(w):
        w = np.clip(w, 0, None); w = w / w.sum()
        combined = np.tensordot(w, base_probs, axes=(0, 0))   # (N, K)
        combined = np.clip(combined, 1e-9, 1.0)
        return -np.mean(np.sum(onehot * np.log(combined), axis=1))
    w0 = np.full(M, 1.0 / M)
    cons = ({'type': 'eq', 'fun': lambda w: w.sum() - 1},)
    bnds = [(0, 1)] * M
    res = minimize(neg_loglik, w0, bounds=bnds, constraints=cons)
    return res.x / res.x.sum()
```

A10.3 Model Selection: WAIC and LOO

For the Bayesian members of the ensemble, model comparison uses the Widely Applicable Information Criterion (WAIC) and Pareto-smoothed leave-one-out cross-validation (PSIS-LOO) from ArviZ, which estimate out-of-sample predictive accuracy directly from the posterior:

```
import arviz as az

def compare_bayesian_models(idata_dict):
    """idata_dict: {name: arviz.InferenceData} with log_likelihood groups."""
    comparison = az.compare(idata_dict, ic="waic", scale="deviance")
    print(comparison)           # ranks models by expected log predictive density
    return comparison
```

A10.4 The Combined Regime Output Contract

Whatever combination method is used, the engine emits a single contract per day: the ensembled regime probability vector, its conformalised prediction set, the dominant model and its weight, the epistemic/aleatoric split, and a model-health flag (BOCPD changepoint, online/batch reconciliation gap, out-of-distribution score). This contract - not any individual model - is what flows into allocation guidance and the Investment Committee artefact.

> *Direction over price. Calibrated probability over point forecast. A documented ensemble over a single black box.*

SECTION A11: DATA REQUIREMENTS AND STRUCTURE

A11.1 Market Data Structure

Field	Type	Description	Example
DateKey	Integer PK	YYYYMMDD integer date key	20250401
Date	Date	Trading date IST	2025-04-01
NiftyClose	Decimal(10,2)	Nifty 50 closing level	22,485.30
NiftyReturn	Decimal(8,6)	Daily log return	0.0042
IndiaVIX	Decimal(5,2)	India VIX closing level	14.85
Advances	Integer	NSE advancing issues	1,243
Declines	Integer	NSE declining issues	856
FII_Equity	Decimal(10,2)	FII net equity flow (₹ Cr)	-485.50
DII_Equity	Decimal(10,2)	DII net equity flow (₹ Cr)	1,250.80
USDINR	Decimal(8,4)	USD/INR closing rate	83.4250
Gilt10Y	Decimal(6,3)	10Y G-Sec yield (%)	7.085

A11.2 Regime Probability Output Structure

Field	Type	Description	Example
DateKey	Integer	YYYYMMDD	20250401
RegimeID	Integer FK	FK to regime dimension	1
Probability	Decimal(6,5)	Ensembled posterior probability	0.52145
Conformal_Lower	Decimal(6,5)	Conformal lower bound (90%)	0.46210
Conformal_Upper	Decimal(6,5)	Conformal upper bound (90%)	0.58320
EpistemicUnc	Decimal(6,5)	Ensemble-disagreement std	0.04125
AleatoricUnc	Decimal(6,5)	Per-member entropy	0.32480
DominantModel	String	Highest-weight ensemble member	RSVAR_v2
Source	String	ModelID generating prediction	ENSEMBLE_v3.2
InferenceTimestamp	Datetime	When prediction made	2025-04-01 18:30:15

A11.3 Required Macro and Flow Indicators

The following indicators feed the regime engine:

- RBI Policy: Repo rate, reverse repo, CRR, stance (hawkish/dovish/neutral)
- Growth: India IIP, PMI Manufacturing, PMI Services, GDP nowcast
- Inflation: CPI headline, CPI core, WPI

- Liquidity: Banking system liquidity, MIBOR, OIS curve
- External: Crude oil (Brent), Gold, DXY, US 10Y, EM equities relative
- Flow: FII equity, FII debt, DII equity, mutual fund monthly SIP inflow, mutual fund inflow by category
- Sentiment: India VIX, Put-Call Ratio (Nifty options), advance-decline, % stocks above 50/100/200 DMA

SECTION A12: SCENARIO ANALYSIS FRAMEWORK

A12.1 Indian Macro Regime Scenarios

Standard scenarios for testing regime engine robustness:

Scenario	RBI Policy	Growth	Expected Regime Response
RBI Hawkish Surprise	+50bps	Steady	Transition toward Risk-Off / Late-Cycle
RBI Dovish Pivot	-50bps	Slowing	Transition toward Post-Shock / Risk-On
Inflation Shock	Hold	Stagflation	Risk-Off probability rises
FPI Sudden Stop	Hold	Steady	Risk-Off; INR depreciation amplifies
China Stimulus	Hold	EM Rally	Risk-On probability rises broadly
Global Risk-Off	Hold	Steady	Risk-Off; safe-haven flows out of EM
Election Surprise	Hold	Uncertain	Transitional; high conviction loss

A12.2 Volatility-Regime Scenarios

- Low VIX (<13): Mean-reversion patterns dominate; SIP-driven steady accumulation; regime stickiness high
- Normal VIX (13-18): Balanced regime distribution; transitions moderate
- Elevated VIX (18-24): Regime shifts accelerate; risk-on/risk-off ambiguity increases
- Crisis VIX (>24): Regime probabilities concentrate on Risk-Off; conformal intervals widen; conviction may rise paradoxically

```
def detect_vol_regime(vix_series, lookback=252):
    """Classify VIX regime for scenario tagging."""
    conditions = [
        vix_series < 13,
        (vix_series >= 13) & (vix_series < 18),
        (vix_series >= 18) & (vix_series < 24),
        vix_series >= 24
```

```
    ]
    labels = ['LOW', 'NORMAL', 'ELEVATED', 'CRISIS']
    return np.select(conditions, labels, default='NORMAL')
```

A12.3 Flow-Regime Scenarios

Indian equity markets are uniquely sensitive to flow dynamics. The regime engine must adapt to flow-driven scenarios that decouple from fundamentals:

- FII Selling + DII Buying: Persistent Indian-specific pattern; price impact muted but composition shifts
- SIP Acceleration: Structural buying flow; supports market floor during macro stress
- SIP Stoppage Wave: Less common but historically signals retail capitulation
- FPI Sudden Stop: Acute, INR-correlated; risk-off probability rises sharply

SECTION A13: MONTE CARLO SIMULATION FOR REGIME-CONDITIONED PORTFOLIOS

A13.1 Regime-Conditioned Path Simulation

Once regime probabilities are estimated, Monte Carlo simulation can project portfolio paths conditioned on a sampled regime trajectory. This produces a regime-aware joint distribution over forward outcomes - exactly the artefact an Investment Committee needs to evaluate a tactical tilt.

```
class RegimeConditionedMC:
    def __init__(self, transition_matrix, regime_returns, regime_vols, K=5):
        self.P = transition_matrix   # (K, K)
        self.mu = regime_returns     # (K,) daily mean returns per regime
        self.sig = regime_vols       # (K,) daily vol per regime
        self.K = K
    def simulate(self, init_regime_dist, horizon=252, n_sims=10000, seed=42):
        rng = np.random.default_rng(seed)
        s0 = rng.choice(self.K, size=n_sims, p=init_regime_dist)
        states = np.zeros((n_sims, horizon), dtype=int); states[:, 0] = s0
        for t in range(1, horizon):
            probs = self.P[states[:, t-1]]
            cum = probs.cumsum(axis=1)
            u = rng.random(n_sims)
            states[:, t] = (u[:, None] < cum).argmax(axis=1)
        mu_t = self.mu[states]; sig_t = self.sig[states]
        eps = rng.standard_normal((n_sims, horizon))
        rets = mu_t + sig_t * eps
        paths = np.exp(np.cumsum(np.log1p(rets), axis=1))
        return paths, states
    def percentile_paths(self, paths, qs=(0.05, 0.25, 0.50, 0.75, 0.95)):
        return {q: np.percentile(paths, q*100, axis=0) for q in qs}
```

A13.2 Regime-Conditioned VaR

```
def regime_var(paths, alpha=0.05):
    """Value-at-Risk on final portfolio value distribution."""
    final = paths[:, -1] - 1.0
    var = np.percentile(final, alpha * 100)
    cvar = final[final <= var].mean()
    return var, cvar
```

A13.3 Communicating MC Outputs to Investment Committee

The Monte Carlo output is most useful when communicated as conditional statements:

- "Conditional on current regime probabilities, the 90% one-year prediction interval for the scheme is +3% to +24%"
- "Probability of negative one-year return: 14%"
- "Worst-case (5th percentile) drawdown over the year: -18%"
- "If regime transitions to Risk-Off within 90 days (probability: 18%), worst-case drawdown rises to -29%"

These statements are testable, defensible, and process-friendly - and they make explicit the conditioning on regime probabilities that makes the engine's value visible to non-quants.

SECTION A14: PROBABILISTIC FORECAST EVALUATION AND PROPER SCORING RULES

A regime engine that emits probabilities cannot be judged by classification accuracy alone. A model that says "90% Risk-On" and a model that says "55% Risk-On" are both "correct" if the market is Risk-On, yet they make very different claims about the world. Evaluating a probabilistic forecaster requires proper scoring rules - loss functions whose expected value is minimised only by reporting the true probability. This section is the evaluation backbone for the entire scoring framework in Part B.

A14.1 Why Accuracy Misleads

Accuracy collapses a probability vector to its arg-max and discards the information that matters most for an Investment Committee: how confident the call was and whether that confidence was justified. Two pathologies follow. First, accuracy rewards overconfidence in easy regimes and is blind to dangerous overconfidence in hard ones. Second, in an imbalanced regime distribution (Risk-On dominates most calendars), a model that always predicts Risk-On scores high accuracy while being useless precisely when it matters - at transitions.

A14.2 The Core Proper Scoring Rules

Scoring Rule	Definition (per forecast)	Properties
Log loss	-log p(true class)	Strictly proper; severely penalises confident errors; unbounded
Brier score	Sum over classes of (p_k - y_k)^2	Strictly proper; bounded [0,2]; decomposes into reliability-resolution-uncertainty
Ranked Probability Score	Sum over thresholds of (CDF_pred - CDF_obs)^2	Strictly proper; rewards forecasts that are close on the ordinal regime scale
CRPS (continuous)	Integral of (F_pred - 1{x>=y})^2 dx	Strictly proper; for continuous forward-return forecasts; generalises MAE

For ordered regimes (Risk-Off → Post-Shock → Transitional → Late-Cycle → Risk-On), the Ranked Probability Score is preferred over log loss because it understands that predicting Late-Cycle when the truth is Risk-On is a smaller error than predicting Risk-Off.

```
import numpy as np

def log_loss(probs, y_true, eps=1e-12):
    p = np.clip(probs[np.arange(len(y_true)), y_true], eps, 1.0)
    return -np.mean(np.log(p))

def brier_score(probs, y_true, K):
```

```
    onehot = np.eye(K)[y_true]
    return np.mean(np.sum((probs - onehot) ** 2, axis=1))

def ranked_probability_score(probs, y_true, K):
    """RPS for ORDERED regime classes (lower is better)."""
    onehot = np.eye(K)[y_true]
    cdf_pred = np.cumsum(probs, axis=1)
    cdf_obs  = np.cumsum(onehot, axis=1)
    return np.mean(np.sum((cdf_pred - cdf_obs) ** 2, axis=1)) / (K - 1)
```

A14.3 The Murphy Decomposition

The Brier score decomposes into three interpretable components (Murphy, 1973): Reliability (are the forecast probabilities calibrated?), Resolution (do the forecasts discriminate between outcomes?), and Uncertainty (the irreducible base-rate variance). The decomposition is Brier = Reliability - Resolution + Uncertainty. A good model has low reliability error (well-calibrated) and high resolution (sharp, discriminating).

```
def brier_decomposition(probs_pos, y, n_bins=10):
    """Reliability-resolution-uncertainty for the binary case (one regime
       vs rest). probs_pos: predicted P(regime); y: 0/1 outcome."""
    base = y.mean()
    bins = np.linspace(0, 1, n_bins + 1)
    rel, res = 0.0, 0.0
    N = len(y)
    for lo, hi in zip(bins[:-1], bins[1:]):
        m = (probs_pos > lo) & (probs_pos <= hi)
        nk = m.sum()
        if nk == 0:
            continue
        pk = probs_pos[m].mean()       # mean forecast in bin
        ok = y[m].mean()               # observed frequency in bin
        rel += nk * (pk - ok) ** 2
        res += nk * (ok - base) ** 2
    rel /= N; res /= N
    unc = base * (1 - base)
    return {'reliability': rel, 'resolution': res,
            'uncertainty': unc, 'brier': rel - res + unc}
```

A14.4 Skill Scores and Benchmarking

Raw scores are hard to interpret in isolation. A skill score expresses improvement over a naive reference forecaster - here, the climatological base rate (the unconditional regime frequencies) or a persistence forecast (tomorrow's regime equals today's). Skill = 1 - Score_model / Score_reference. A positive skill score is the minimum bar for a regime model to earn its place in the ensemble.

```
def skill_score(score_model, score_reference):
    """Positive => model beats the naive reference; 1.0 => perfect."""
    return 1.0 - (score_model / score_reference)

# Reference 1: climatology (unconditional regime frequencies)
# Reference 2: persistence (P(regime_t) = onehot(regime_{t-1}))
# Every ensemble member must beat BOTH references on RPS skill out-of-sample.
```

> *A model that cannot beat persistence on a proper scoring rule has learned nothing about regimes - it has learned that regimes are sticky, which we already knew.*

SECTION A15: HIERARCHICAL, STICKY, AND NONPARAMETRIC REGIME MODELS

The HMM of Section A3 assumes a fixed number of regimes and estimates each scheme or index independently. Both assumptions are limiting. Hierarchical models share statistical strength across related series; sticky and nonparametric models relax the rigid regime-count and regime-duration assumptions. These are the advanced model-building techniques that separate a competent submission from an exceptional one.

A15.1 Hierarchical Pooling Across Schemes and Sectors

A fund house runs many schemes across many sectors. Estimating a separate regime model for each throws away the fact that they share macro drivers; estimating one pooled model ignores genuine differences. Hierarchical Bayesian modelling is the principled middle path: scheme-level parameters are drawn from a common hyper-distribution, so data-rich schemes inform data-poor ones (partial pooling).

```
import pymc as pm
import numpy as np

def hierarchical_regime_means(returns_by_scheme, K=5):
    """Partial-pooling of regime-conditional means across S schemes.
       returns_by_scheme: list of (T_s,) arrays; regimes assumed pre-decoded."""
    S = len(returns_by_scheme)
    with pm.Model() as model:
        # global hyper-priors over regime means and spreads
        mu_global = pm.Normal('mu_global', 0.0, 0.02, shape=K)
        tau       = pm.HalfNormal('tau', 0.01, shape=K)
        # scheme-level regime means shrink toward the global mean
        mu_scheme = pm.Normal('mu_scheme', mu=mu_global, sigma=tau, shape=(S, K))
        sigma     = pm.HalfNormal('sigma', 0.03, shape=(S, K))
        for s in range(S):
            r, z = returns_by_scheme[s]   # z = decoded regime index per t
            pm.Normal(f'obs_{s}', mu=mu_scheme[s][z], sigma=sigma[s][z],
                      observed=r)
```

```
    trace = pm.sample(1500, tune=1500, target_accept=0.95, chains=4)
  return model, trace
```

A15.2 The Sticky HDP-HMM

How many regimes are there? Five is a modelling convenience, not a law of nature. The Hierarchical Dirichlet Process HMM (HDP-HMM; Teh et al., 2006) lets the data determine the number of states, with a countably infinite prior truncated in practice. The vanilla HDP-HMM, however, over-segments rapidly switching data into too many short-lived states. The sticky HDP-HMM (Fox, Sudderth, Jordan, Willsky, 2008) adds a self-transition bias parameter κ that encodes the prior belief that regimes persist - exactly right for financial markets.

```
# Sticky transition prior intuition (Fox et al. 2008):
#   beta ~ GEM(gamma)                    # global stick-breaking weights
#   pi_k ~ DP(alpha + kappa, (alpha*beta + kappa*delta_k)/(alpha + kappa))
# The kappa term adds extra prior mass to the self-transition pi_{kk},
# biasing toward persistent regimes and suppressing spurious short states.

# Practical route: use a large-K weak-limit approximation with an
# asymmetric Dirichlet transition prior that puts heavy mass on the diagonal.
import numpy as np

def sticky_transition_prior(K, alpha=1.0, kappa=12.0):
  """Asymmetric Dirichlet concentration matrix approximating stickiness."""
  return alpha * np.ones((K, K)) + kappa * np.eye(K)
```

A15.3 Explicit-Duration Models (HSMM)

A standard HMM implies geometrically distributed regime durations - the probability of staying falls off at a constant rate. Empirically, Indian market regimes do not behave this way: a young bull market is more likely to persist than a geometric law would predict. Hidden Semi-Markov Models (HSMM) replace the geometric duration with an explicit duration distribution (negative binomial, Poisson, or empirical), giving a far better fit to observed regime-length histograms.

- Geometric (HMM-implied): memoryless; constant exit hazard - usually too short on average
- Negative binomial: flexible over-dispersed durations; good default for regime lengths
- Empirical/non-parametric: duration distribution estimated directly from labelled history

The candidate should fit regime-duration histograms from the decoded HMM, test them against a geometric null, and - if the null is rejected - upgrade to an HSMM duration model. Evidence of having checked the duration assumption is itself a scored component of Bayesian rigour.

SECTION A16: REGIME-AWARE PORTFOLIO CONSTRUCTION AND ALLOCATION

Regime probabilities are an input, not an output. Their value is realised only when they change a portfolio decision. This section bridges the engine to the allocation overlay that the backtesting engine (Deliverable 4) evaluates, and it grounds the Allocation Logic scoring category in Part B.

A16.1 From Regime Probabilities to Expected Returns and Covariances

Given regime probabilities and regime-conditional return moments (from the RS-VAR of Section A8), the unconditional forward moments are a probability-weighted mixture. The mixture covariance correctly includes both the average within-regime covariance and the dispersion of regime-conditional means - the second term is exactly the extra risk that regime uncertainty injects and that single-regime models miss.

```
import numpy as np

def mixture_moments(regime_probs, regime_means, regime_covs):
    """regime_probs: (K,); regime_means: (K, n_assets);
       regime_covs: (K, n_assets, n_assets). Returns mixture mean & cov."""
    w = regime_probs / regime_probs.sum()
    mu = np.tensordot(w, regime_means, axes=(0, 0))          # (n_assets,)
    within = np.tensordot(w, regime_covs, axes=(0, 0))       # E[cov | regime]
    diff = regime_means - mu                                 # (K, n_assets)
    between = np.einsum('k,ki,kj->ij', w, diff, diff)        # cov of regime means
    return mu, within + between
```

A16.2 Regime Views via Black-Litterman

The Black-Litterman framework blends a market-equilibrium prior with subjective views. Regime probabilities map naturally onto views: a high Risk-Off probability becomes a negative-tilt view on high-beta sleeves with a confidence set by the engine's conviction (1 minus the conformal-set width). This keeps the allocation anchored to equilibrium while letting calibrated regime conviction - not point forecasts - move the tilts.

```
def black_litterman_regime(w_mkt, Sigma, P, Q, Omega, tau=0.05, delta=2.5):
    """w_mkt: market weights; Sigma: covariance; P,Q: regime views;
       Omega: view-uncertainty (diagonal, from conformal-set width)."""
    pi = delta * Sigma @ w_mkt                    # implied equilibrium returns
    tS = tau * Sigma
    A_ = np.linalg.inv(tS) + P.T @ np.linalg.inv(Omega) @ P
    b_ = np.linalg.inv(tS) @ pi + P.T @ np.linalg.inv(Omega) @ Q
    mu_bl = np.linalg.solve(A_, b_)               # posterior expected returns
    w_opt = np.linalg.solve(delta * Sigma, mu_bl)    # unconstrained optimal weights
    return mu_bl, w_opt
```

A16.3 Sizing Under Regime Uncertainty

How large a tilt should a given conviction justify? The fractional-Kelly heuristic scales the tilt by the edge over variance, then deliberately under-bets (typically half-Kelly) to respect parameter

uncertainty. Crucially, the tilt is multiplied by the engine's conviction, so an uncertain regime call produces a small tilt and a confident one a larger tilt - the position size inherits the calibration of the probability.

```
def conviction_scaled_tilt(edge, variance, conviction, kelly_fraction=0.5,
                           max_tilt=0.10):
    """edge: regime-conditional expected excess return; variance: its variance;
       conviction: 1 - conformal_set_width in [0,1]."""
    raw = kelly_fraction * (edge / max(variance, 1e-9))
    tilt = raw * conviction
    return float(np.clip(tilt, -max_tilt, max_tilt))
```

A16.4 Constraints, Turnover, and SEBI Reality

A deployable overlay must respect the operating constraints of an Indian long-only scheme:

- Long-only and category constraints - a large-cap scheme cannot tilt outside its SEBI-defined universe; tilts are expressed within the permitted sleeve
- Turnover budget - regime calls flip; without a turnover penalty the overlay churns and bleeds cost. Penalise L1 distance from current weights and apply a no-trade band around small tilts
- Transaction costs and impact - mid- and small-cap tilts carry impact cost; size tilts against liquidity, not just conviction
- Hysteresis - require regime probability to cross a band (e.g. 0.60 to enter, 0.40 to exit) before acting, so the overlay does not whipsaw on a single noisy day

```
def apply_no_trade_band(target_w, current_w, band=0.015):
    """Suppress tiny rebalances to control turnover."""
    delta = target_w - current_w
    delta[np.abs(delta) < band] = 0.0
    return current_w + delta
```

SECTION A17: EXPLAINABILITY, ATTRIBUTION, AND MODEL DIAGNOSTICS

A regime call that cannot be explained cannot be defended to an Investment Committee or a SEBI inspection. Explainability is not an optional add-on; it is a first-class deliverable and a scored component of audit-defensibility. This section covers attribution for both the feature-based models and the foundation-model head.

A17.1 SHAP for Regime Classifiers

SHAP (SHapley Additive exPlanations) assigns each feature a contribution to a specific regime prediction, grounded in cooperative game theory and satisfying local accuracy and consistency. For a regime classifier, SHAP answers the Investment Committee's first question: why does the engine think we are in Risk-Off today?

```
import shap
```

```
def explain_regime_call(model, X_background, x_today, feature_names):
    """KernelSHAP attribution for a single day's regime probabilities."""
    explainer = shap.KernelExplainer(model.predict, X_background)
    shap_values = explainer.shap_values(x_today, nsamples=200)
    # shap_values[k]: contributions to regime k for x_today
    contribs = dict(zip(feature_names, shap_values))
    return explainer, shap_values, contribs

# Report the top-5 positive and top-5 negative contributors to the dominant
# regime as a plain-language rationale line in the Investment Committee artefact.
```

A17.2 Permutation Importance and Partial Dependence

Global explanations complement local ones. Permutation importance measures how much a proper score degrades when a feature is shuffled - a model-agnostic, leakage-aware importance ranking. Partial dependence and accumulated local effects show the marginal shape of the relationship between a feature (say, India VIX) and a regime probability, surfacing non-monotonicities that a linear intuition would miss.

```
def permutation_importance_rps(model, X, y, K, n_repeats=10, seed=42):
    from numpy.random import default_rng
    rng = default_rng(seed)
    base = ranked_probability_score(model.predict(X), y, K)
    importances = {}
    for j in range(X.shape[1]):
        drops = []
        for _ in range(n_repeats):
            Xp = X.copy()
            Xp[:, j] = rng.permutation(Xp[:, j])
            drops.append(ranked_probability_score(model.predict(Xp), y, K) - base)
        importances[j] = float(np.mean(drops))   # larger => more important
    return importances
```

A17.3 Diagnosing the Foundation-Model Head

Foundation-model embeddings are opaque by construction. Three diagnostics restore interpretability: attention-weight inspection (which historical windows the model attends to when forming today's embedding), embedding-space clustering coloured by decoded regime (do regimes separate in embedding space?), and probing classifiers (can a simple linear probe recover known features from the embedding, confirming the embedding encodes them?). The candidate must show at least one such diagnostic for each foundation model used.

A17.4 Feature and Prediction Stability Monitoring

A model can be perfectly explainable yet quietly drifting. Population Stability Index (PSI) tracks distribution shift in each input feature against its training distribution; a regime-prediction

stability check tracks how often the dominant regime flips day-to-day. Sudden jumps in either are model-health alarms that should gate any allocation action.

```
def population_stability_index(expected, actual, n_bins=10):
    """PSI > 0.25 conventionally signals significant distribution shift."""
    qs = np.quantile(expected, np.linspace(0, 1, n_bins + 1))
    qs[0], qs[-1] = -np.inf, np.inf
    e = np.histogram(expected, qs)[0] / len(expected)
    a = np.histogram(actual, qs)[0] / len(actual)
    e = np.clip(e, 1e-6, None); a = np.clip(a, 1e-6, None)
    return float(np.sum((a - e) * np.log(a / e)))
```

SECTION A18: PRODUCTION DEPLOYMENT, MLOPS, AND MODEL GOVERNANCE

The gap between a notebook that fits a regime model and an engine a fund house runs every trading day is the gap this section closes. It is also where SEBI's direction of travel - toward documented, governed, auditable models - becomes a concrete engineering checklist.

A18.1 The Two-Speed Serving Architecture

As established in Section A9, the engine runs at two speeds. The nightly batch process refits the Bayesian HMM and RS-VAR, recomputes ensemble weights, recalibrates the conformal layer, and writes the reference posterior. The intraday online process runs the particle filter and BOCPD against the live feed, tracking the state between refits in milliseconds. The serving contract is a single endpoint that returns the combined regime output contract (Section A10.4) with a timestamp and the model versions that produced it.

```
# Logical service layout (FastAPI sketch - not Power-anything)
# POST /regime/score  -> { date, regime_probs, conformal_set, dominant_model,
#                          epistemic, aleatoric, health_flags, model_versions }
# GET  /regime/health -> { last_batch_refit, online_batch_gap, psi_max,
#                          changepoint_active, coverage_30d }
#
# Batch job (cron, ~02:00 IST): refit -> validate -> register -> promote
# Online loop (per tick/EOD bar): particle_filter.step(); bocpd.update();
#                                 reconcile_with_batch(); emit_contract()
```

A18.2 Model Registry, Versioning, and Lineage

Every promoted model carries an immutable version, the exact training-data snapshot hash, its priors and hyperparameters, its calibration and proper-score metrics, and the code commit that produced it. The combined output contract records which versions generated each day's call. This is what makes a regime call reproducible months later when an inspector asks why the engine said Risk-Off on a given date.

- Data lineage - point-in-time snapshot hash so the exact inputs are recoverable

- Model lineage - registry entry with priors, diagnostics, scores, and code commit
- Decision lineage - the output contract plus the SHAP rationale that fed the allocation

A18.3 Drift Detection and Retraining Triggers

Retraining on a fixed calendar is wasteful when nothing has changed and dangerous when the world has moved faster than the calendar. The engine retrains on triggers: a PSI breach on key features, a sustained conformal-coverage shortfall, a BOCPD changepoint, or an online/batch reconciliation gap beyond tolerance. Every trigger and every retrain is logged.

```
def retraining_trigger(coverage_30d, target_cov, psi_max, recon_gap,
                       changepoint_active):
    reasons = []
    if coverage_30d < target_cov - 0.05: reasons.append('coverage_shortfall')
    if psi_max > 0.25:                   reasons.append('feature_drift')
    if recon_gap > 0.15:                 reasons.append('online_batch_divergence')
    if changepoint_active:               reasons.append('regime_changepoint')
    return (len(reasons) > 0, reasons)
```

A18.4 Model Risk Governance

Borrowing the discipline of bank model-risk management (and anticipating SEBI's direction), the engine sits inside a governance frame: an independent validation of the model before first deployment, a champion-challenger process where a candidate model must beat the incumbent on out-of-sample proper scores before promotion, a documented model-risk tier reflecting how much capital the model influences, and periodic revalidation. The candidate need not build the org process, but must produce the artefacts it consumes: the model card, the validation report, and the monitoring metric definitions.

SECTION A19: DATA ENGINEERING FOR INDIAN MARKET REGIME PIPELINES

Model quality is capped by data quality, and Indian market data has specific traps. This section is the practical data-engineering training that prevents the most common - and most damaging - silent errors in a regime pipeline: look-ahead bias and survivorship bias.

A19.1 Point-in-Time Data and Look-Ahead Bias

The single most destructive error in financial modelling is using information that was not available at the decision time. Macro releases are revised; index constituents change; corporate actions are applied retroactively in many data vendors. A point-in-time discipline records what was known as of each date and refuses the model any later-vintage value. A regime model that quietly trains on revised GDP or restated earnings will look brilliant in backtest and fail in production.

- Macro vintages - use first-print values with their actual release dates, not the latest revised series

- Index membership - reconstruct the Nifty/Midcap constituent set as of each date; do not apply today's membership backward
- Feature timestamps - lag any feature by its true availability (EOD breadth is known only after close; SIP totals only after AMFI's monthly release)

```
import pandas as pd

def enforce_pit(feature_df, release_lag_days: dict):
    """Shift each feature forward by its true availability lag so the model
       can never see a value before it was actually published."""
    out = feature_df.copy()
    for col, lag in release_lag_days.items():
        if col in out.columns:
            out[col] = out[col].shift(lag)
    return out

# Example lags (trading days): macro prints 1, AMFI SIP totals 5,
# index reconstitution effective dates per NSE circular.
```

A19.2 Corporate Actions and Survivorship Bias

Prices must be adjusted for splits, bonuses, and dividends to be comparable across time, but the adjustment must be applied as-of, not retroactively leaked. Survivorship bias - building a universe from only the stocks that exist today - systematically excludes the IL&FS-style casualties that define Risk-Off regimes, biasing the engine toward optimism. The training universe must include delisted and merged names with their real exit dates.

A19.3 Alignment, Gaps, and the Trading Calendar

Indian markets observe a distinct holiday calendar; global features (US 10Y, DXY, crude) trade on a different calendar. Naive merging creates phantom gaps and forward-fill leakage. The pipeline aligns everything to the NSE trading calendar, forward-fills global features only up to the Indian close, and flags any series with excessive gaps before it reaches the model.

```
def align_to_nse_calendar(frames: dict, nse_sessions: pd.DatetimeIndex):
    """frames: {name: Series}; reindex all to NSE sessions with controlled ffill."""
    aligned = {}
    for name, s in frames.items():
        a = s.reindex(nse_sessions)
        # ffill global/macro series at most 3 sessions; never ffill returns
        if name not in ('nifty_return', 'fii_equity', 'dii_equity'):
            a = a.ffill(limit=3)
        aligned[name] = a
    return pd.DataFrame(aligned)
```

A19.4 Feature Store and Reproducibility

A feature store separates feature computation from model training, so the same point-in-time feature definitions serve both the nightly batch refit and the intraday online loop - eliminating training/serving skew, a notorious source of silent production failure. Each feature carries an owner, a definition, a release lag, and a version. Snapshotting the feature store (with DVC or equivalent) makes every backtest exactly reproducible.

SECTION A20: WORKED END-TO-END EXAMPLE - A SINGLE REGIME CALL

This section ties the entire stack together with a concrete, illustrative walkthrough of one day's regime call - the workflow the candidate must be able to demonstrate live in the Day 15 video. Figures below are illustrative, chosen to show the mechanics, not to assert a historical fact.

A20.1 The Inputs

Suppose it is a trading day in mid-2025. The point-in-time feature vector, assembled by the pipeline of Section A19, shows: Nifty above its 200-DMA but with narrowing breadth (42% of constituents above their 50-DMA, down from 68% a month earlier), India VIX at 16.5 with a positive 5-day change, modest FII outflows, steady DII inflows on continued SIP momentum, and a flat INR. The signature is ambiguous - trend up, breadth deteriorating - the classic Late-Cycle versus Transitional boundary.

A20.2 The Member Models Speak

Model	Dominant Regime	P(dominant)	Note
Bayesian HMM	Late-Cycle	0.48	Wide posterior; credible interval 0.36-0.59
RS-VAR	Late-Cycle	0.55	Breadth + flow divergence loads on Late-Cycle
BNN ensemble	Transitional	0.41	High epistemic uncertainty (members disagree)
Foundation head	Late-Cycle	0.51	Embedding near 2018-Q3 analogues

A20.3 Ensembling and Calibration

Stacking weights, learned out-of-fold, currently favour the RS-VAR and foundation head (they have led recent out-of-sample proper scores). Bayesian model averaging tilts the same way. The combined contract emits: Late-Cycle 0.52, Transitional 0.27, Risk-On 0.14, Risk-Off 0.05, Post-Shock 0.02. The conformalised prediction set at 90% coverage is {Late-Cycle, Transitional} - the engine is honestly saying the market is in one of these two adjacent states and is not Risk-On. Epistemic uncertainty is elevated (members disagreed), aleatoric is moderate.

A20.4 Health Checks and the Decision

BOCPD shows no active changepoint; the online particle filter agrees with the last batch posterior within tolerance (reconciliation gap 0.06); PSI on all features is below 0.25. No health alarm. The SHAP rationale attributes the Late-Cycle call primarily to the breadth collapse and the small-cap-versus-large-cap divergence, secondarily to the VIX uptick. Conviction - one minus the conformal-set width - is moderate, so the allocation overlay applies a small de-risking tilt within the scheme's sleeve (trim high-beta, add quality), sized by the conviction-scaled rule of Section A16.3 and passed through the no-trade band and hysteresis bands. The Investment Committee artefact records the contract, the rationale line, the model versions, and the data snapshot hash.

> *No point forecast was made. A calibrated, explainable, audit-defensible probability over direction was produced, and it changed the portfolio by exactly as much as the conviction justified - no more.*

SECTION A21: SELF-ASSESSMENT EXERCISES

The following exercises map to the model stack and are graded implicitly through the deliverables. Early career talent should be able to answer each before submission.

Foundations and evaluation

1. Explain, in two sentences each, why log loss and the Ranked Probability Score can disagree on which of two regime models is better. Which would you trust for ordered regimes, and why?
2. Construct a forecaster that achieves 80% accuracy on Indian equity regimes yet has negative skill on RPS against persistence. What does this reveal about accuracy as a metric?
3. Decompose your best model's Brier score into reliability, resolution, and uncertainty. Which component does conformal calibration improve, and which does it leave untouched?

Modelling

4. Fit a 5-state HMM and inspect the implied regime-duration distribution. Test it against a geometric null. If rejected, justify whether an HSMM upgrade is warranted.
5. From the RS-VAR posterior, plot the regime-conditional impulse response of Nifty returns to a one-standard-deviation FII-outflow shock. Explain why the Risk-Off response differs from the Risk-On response.
6. Show that your ensemble beats every individual member on out-of-sample RPS. If it does not, diagnose whether the cause is correlated members, leakage in the stacking split, or a dominant single model.

Sequential and online

7. Run BOCPD over 2018-2020. Confirm it fires on the IL&FS and COVID breaks and report its false-positive rate on routine 2019 data. Tune the hazard rate to balance the two.
8. Quantify the online/batch reconciliation gap over a held-out month. At what gap threshold would you trigger an intraday retrain, and why?

Deployment and defensibility

9. For a single chosen date, produce the full combined output contract and the SHAP rationale. Could an inspector reconstruct this call from your registry and data snapshot alone?
10. Identify one place in your pipeline where look-ahead bias could enter undetected, and show the point-in-time control that prevents it.

SECTION A22: ROBUST BACKTESTING METHODOLOGY AND OVERFITTING CONTROL

A backtest is the easiest thing in quantitative finance to get spectacularly, expensively wrong. A regime overlay that looks superb in-sample is the default outcome, not the exception, because the researcher has - usually unknowingly - tuned the strategy to the very data used to evaluate it. This section is the discipline that makes the backtesting engine (Deliverable 4) trustworthy, and it directly protects the credibility of every claim the candidate makes about the overlay's Information Ratio.

A22.1 Why Standard Cross-Validation Fails on Financial Series

k-fold cross-validation assumes the samples are independent and exchangeable. Financial features are neither: a 21-day momentum feature on adjacent days shares 20 of its 21 inputs, and a forward return label spans days that appear in other folds. The result is information leakage between train and test folds, which inflates measured performance. Two corrections are mandatory: purging and embargoing.

- Purging - remove from the training set any sample whose label window overlaps the test set, so a label's future is never used to predict its own past
- Embargo - additionally drop a buffer of training samples immediately after each test fold, to neutralise the serial correlation that bleeds across the fold boundary

```
import numpy as np

def purged_kfold_indices(n, n_splits=5, label_horizon=21, embargo=10):
    """Yield (train_idx, test_idx) with purging and an embargo, for time
       series where each label looks 'label_horizon' steps into the future."""
    fold_sizes = np.full(n_splits, n // n_splits)
    fold_sizes[: n % n_splits] += 1
    bounds, start = [], 0
    for fs in fold_sizes:
        bounds.append((start, start + fs)); start += fs
    all_idx = np.arange(n)
    for (t0, t1) in bounds:
        test_idx = all_idx[t0:t1]
        # purge label-overlap region and apply embargo after the test fold
        purge_lo = max(0, t0 - label_horizon)
        purge_hi = min(n, t1 + label_horizon + embargo)
        train_mask = np.ones(n, dtype=bool)
        train_mask[purge_lo:purge_hi] = False
        yield all_idx[train_mask], test_idx
```

A22.2 Walk-Forward Evaluation

The most honest backtest mimics live operation: train on the past, predict the next block, roll forward, never look back. Walk-forward (expanding or rolling window) is the default evaluation for the regime overlay because it respects the arrow of time exactly as production does. The candidate must report walk-forward results, not a single in-sample fit, and should show both expanding-window and rolling-window variants to expose any dependence on the training-window length.

```
def walk_forward(model_factory, X, y, train_min=750, test=63, expanding=True):
    """Roll through time: fit on history, score the next block, advance."""
    results = []
    start = train_min
    while start + test <= len(X):
        if expanding:
            tr = slice(0, start)
        else:
            tr = slice(start - train_min, start)
        te = slice(start, start + test)
        model = model_factory()
        model.fit(X[tr], y[tr])
        preds = model.predict(X[te])
        results.append((start, preds, y[te]))
        start += test
    return results
```

A22.3 The Multiple-Testing Problem and the Deflated Sharpe Ratio

If a researcher tries 50 feature sets, 50 priors, and 10 tilt rules, some configuration will post a high backtest Sharpe by chance alone. Reporting that lucky configuration's Sharpe as if it were the only one tried is the central sin of backtest overfitting. The Deflated Sharpe Ratio (Bailey & Lopez de Prado, 2014) discounts the observed Sharpe for the number of trials, the length of the track record, and the skewness and kurtosis of returns, yielding the probability that the true Sharpe exceeds zero.

```
from scipy.stats import norm
import numpy as np

def deflated_sharpe_ratio(sharpe, n_obs, n_trials, skew=0.0, kurt=3.0):
    """Probability the true Sharpe > 0 after correcting for selection.
       sharpe: best observed (per-observation) Sharpe; n_trials: configs tried."""
    # expected maximum Sharpe under the null of zero true skill (n_trials draws)
    e_max = (np.sqrt(2 * np.log(n_trials)) if n_trials > 1 else 0.0)
    sr_std = np.sqrt((1 - skew * sharpe + (kurt - 1) / 4 * sharpe ** 2)
                     / (n_obs - 1))
    z = (sharpe - e_max * sr_std) / max(sr_std, 1e-12)
    return float(norm.cdf(z))   # ~1 => robust; ~0.5 or below => likely luck
```

A22.4 The Probability of Backtest Overfitting

The Probability of Backtest Overfitting (PBO; Bailey, Borwein, Lopez de Prado, Zhu, 2017) uses combinatorially symmetric cross-validation: repeatedly split the trials into in-sample and out-of-sample halves and measure how often the in-sample best performer underperforms the median out-of-sample. A high PBO means the selection process is mostly fitting noise. The candidate should report PBO for the overlay configuration they ultimately choose.

The practical discipline that follows from all of the above:

11. Decide the evaluation protocol (walk-forward, purged CV, the proper scoring rule) before looking at any out-of-sample result.
12. Count and log every configuration tried; do not quietly discard failures.
13. Report the deflated Sharpe and PBO alongside the headline Information Ratio, not as an afterthought.
14. Reserve a genuinely untouched holdout period and look at it exactly once, at the very end.

A22.5 Regime-Specific Backtesting Cautions

- Regime label leakage - if HMM-decoded labels are fit on the full history and then used to train a supervised classifier, the labels themselves contain future information; decode labels within each walk-forward window only
- Conviction-conditioned reporting - report overlay performance separately for high- and low-conviction days; a strategy that only works on high-conviction calls is fine, but the candidate must show it explicitly
- Cost realism - apply Indian-market-realistic impact and turnover costs, especially for any mid/small-cap tilt; a costless backtest is not a backtest
- Crisis attribution - decompose total performance into the Part C crisis episodes; a strategy whose entire edge comes from one lucky COVID call has not demonstrated a repeatable regime skill

The purpose of a backtest is not to prove the strategy works; it is to find the conditions under which it fails. A backtest that only flatters the strategy has not been run honestly.

SECTION A23: GLOSSARY AND QUICK REFERENCE

A consolidated reference for the terminology used throughout this guide. Early career talent should be fluent in every term before the Day 15 demonstration.

A23.1 Statistical and Bayesian Terms

Term	Definition
Prior	Probability distribution encoding beliefs about a parameter before observing data

Term	Definition
Posterior	Updated distribution over a parameter after combining prior and likelihood via Bayes' theorem
Likelihood	Probability of the observed data as a function of the parameters
Credible interval	Bayesian interval containing the parameter with a stated posterior probability (e.g. 95%)
Conjugate prior	Prior whose posterior is in the same family, enabling closed-form updates
MCMC	Markov Chain Monte Carlo; sampling-based approximation of an intractable posterior
NUTS	No-U-Turn Sampler; an adaptive Hamiltonian Monte Carlo algorithm
R-hat	Gelman-Rubin convergence diagnostic; values near 1.00 indicate converged chains
ESS	Effective Sample Size; the number of effectively independent posterior draws
Divergence	An HMC trajectory error signalling a mis-specified or hard-to-sample posterior
Variational inference	Approximate posterior inference by optimising a tractable distribution toward the true posterior
WAIC / PSIS-LOO	Information criteria estimating out-of-sample predictive accuracy from the posterior

A23.2 Regime-Modelling Terms

Term	Definition
HMM	Hidden Markov Model; observations generated by an unobserved Markov chain of states
Transition matrix	Matrix of probabilities of moving from each regime to each other regime
Emission distribution	Distribution of observed features conditional on the current regime
Baum-Welch	Expectation-Maximisation algorithm for fitting HMM parameters
Viterbi	Algorithm recovering the single most likely hidden state sequence
RS-VAR	Regime-Switching Vector Autoregression; regime-conditional multivariate dynamics
HSMM	Hidden Semi-Markov Model; HMM with an explicit (non-geometric) regime-duration distribution
HDP-HMM	Hierarchical Dirichlet Process HMM; nonparametric model inferring

Term	Definition
	the number of regimes
Sticky HDP-HMM	HDP-HMM with a self-transition bias favouring persistent regimes
Smoothed probability	P(regime at t | the entire observed history)
Particle filter	Sequential Monte Carlo method for online posterior state estimation
BOCPD	Bayesian Online Changepoint Detection; online posterior over time since the last break

A23.3 Calibration, Evaluation, and Deployment Terms

Term	Definition
Calibration	Property that forecast probabilities match observed long-run frequencies
Conformal prediction	Model-agnostic method producing prediction sets with finite-sample coverage guarantees
APS	Adaptive Prediction Sets; an efficient conformal classification scheme
EnbPI / ACI	Ensemble Batch Prediction Intervals / Adaptive Conformal Inference; shift-robust conformal methods
ECE	Expected Calibration Error; average gap between confidence and accuracy across bins
Proper scoring rule	Loss minimised in expectation only by reporting the true probability (e.g. log loss, Brier, RPS)
RPS	Ranked Probability Score; proper score respecting the ordering of regimes
CRPS	Continuous Ranked Probability Score; proper score for continuous forecasts
Skill score	Relative improvement of a model's score over a naive reference forecaster
Epistemic uncertainty	Reducible model uncertainty, shrinkable with more data
Aleatoric uncertainty	Irreducible noise in the data-generating process
PSI	Population Stability Index; a measure of input-feature distribution shift
Deflated Sharpe	Sharpe ratio corrected for the number of trials and return non-normality
PBO	Probability of Backtest Overfitting

A23.4 Indian Market and Regulatory Terms

Term	Definition
SEBI	Securities and Exchange Board of India; the securities-market regulator
AMFI	Association of Mutual Funds in India; the industry body
AMC	Asset Management Company; the entity managing mutual fund schemes
SIP	Systematic Investment Plan; recurring retail investment, a structural flow into equities
FII / FPI	Foreign Institutional / Portfolio Investor flows
DII	Domestic Institutional Investor flows (mutual funds, insurers)
TER	Total Expense Ratio; the annual cost charged to a scheme
Risk-O-Meter	SEBI-mandated monthly scheme risk classification
India VIX	NSE's implied-volatility index, a market fear gauge
Breadth	Share of constituents participating in a move (e.g. % above 50-DMA)
Categorisation	SEBI rules restricting each scheme to a defined investment universe

SECTION A24: MACRO TRANSMISSION AND INDIAN REGIME DRIVERS

A regime engine is only as good as its understanding of what actually moves Indian equity regimes. This section is the macro-economic training that turns raw indicator feeds into informed features and priors. The candidate who treats macro series as anonymous columns will build a fragile model; the candidate who understands the transmission mechanisms will know which features matter in which regime and why.

A24.1 The Rate-Growth-Inflation Nexus

The Reserve Bank of India sets the repo rate to balance growth and inflation, and that single lever propagates through the equity market along several channels. Higher rates raise the discount rate applied to future earnings (compressing valuations, hurting long-duration growth stocks most), raise the cost of leverage (hurting capital-intensive and financial-sector earnings differently), and shift the relative attractiveness of debt versus equity. The regime engine should read not just the level of rates but the direction and surprise of policy relative to expectations - a hold can be hawkish if a cut was expected.

Macro Shift	Equity Transmission	Regime Tendency
Rate hike (hawkish surprise)	Discount rate up; valuations compress; growth de-rates	Toward Late-Cycle / Risk-Off
Rate cut (dovish surprise)	Discount rate down; valuations expand; rate-sensitives rally	Toward Post-Shock / Risk-On
Inflation surprise up	Real-rate uncertainty; margin pressure; policy risk	Toward Risk-Off / Transitional
Growth surprise up (IIP/PMI)	Earnings-revision tailwind; broad participation	Toward Risk-On
Liquidity tightening	Funding stress; small-cap and NBFC pressure	Toward Late-Cycle / Risk-Off

A24.2 The Flow Channel - India's Distinctive Driver

More than most markets, Indian equities are shaped by the tug-of-war between foreign and domestic flows. FPI flows are sentiment- and dollar-sensitive and tend to reverse sharply in global risk-off episodes; domestic SIP-driven DII flows are structural, slow-moving, and largely price-insensitive, acting as a shock absorber that defends the market floor. The regime engine must feature both, because the same Nifty move means different things depending on who is driving it: a flat index masking heavy FPI selling absorbed by DIIs is a very different regime from a flat index on balanced flows.

- FPI sudden stop - acute, INR-correlated, raises Risk-Off probability sharply; the 2013 Taper Tantrum is the archetype

- SIP momentum - rising monthly SIP totals support the floor and lengthen Risk-On regimes; stalling SIPs are an early Late-Cycle tell
- Flow divergence (FPI out, DII in) - a persistent Indian pattern that mutes price impact while shifting market composition under the surface

```
import numpy as np
import pandas as pd

def flow_regime_features(df: pd.DataFrame) -> pd.DataFrame:
    """Construct flow-channel features that distinguish who is driving price."""
    f = pd.DataFrame(index=df.index)
    f['fpi_z_60d'] = (df['FII_Equity'] - df['FII_Equity'].rolling(60).mean()) / df['FII_Equity'].rolling(60).std()
    f['sip_momentum'] = df['SIP_Monthly'].pct_change(3)          # 3-month trend
    f['flow_divergence'] = np.sign(df['DII_Equity']) * (df['DII_Equity'] > 0) * (df['FII_Equity'] < 0)              # DII-in, FPI-out
    f['absorption_ratio'] = df['DII_Equity'] / (df['FII_Equity'].abs() + 1e-9)
    return f
```

A24.3 Global Linkages

India is not an island. Three external variables condition the domestic regime with material reliability: the US 10-year yield (the global discount-rate anchor and a key FPI-flow driver), the US dollar index (a strong dollar pressures emerging-market flows and the rupee), and crude oil (India is a large net importer, so a crude spike worsens the trade balance, pressures the rupee, and stokes inflation). A regime engine that ignores these will mis-read EM-specific stress - the kind that hides in benign-looking Nifty prints, as in 2013.

Global Variable	Channel into Indian Equities	Watch For
US 10Y yield	Global discount rate; FPI allocation shifts	Sharp rises draining EM flows
US Dollar (DXY)	Rupee pressure; EM risk appetite	Strong-dollar episodes coinciding with FPI outflows
Brent crude	Import bill, trade balance, inflation, rupee	Spikes that pressure INR and CPI together
Global volatility (VIX)	Risk-appetite spillover into India VIX	Co-movement breaks during India-specific events

A24.4 Translating Macro into Priors and Conditioning

Macro understanding enters the Bayesian engine in two principled places. First, as informative priors: knowledge that risk-off regimes are typically shorter and more violent than risk-on regimes shapes the duration and transition priors of Sections A3 and A15. Second, as conditioning features in the RS-VAR (Section A8), where the macro state alters the regime-

conditional dynamics - the same FPI-outflow shock propagates differently when crude is spiking and the rupee is weak than when both are calm. The candidate should document, for each macro feature, the economic rationale for its inclusion and the regime it most informs; an undocumented feature is a liability under inspection, not an asset.

> *Macro features are not free variables to be thrown at a model. Each earns its place through a transmission mechanism the analyst can name and defend.*

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Early career talent may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: REGIME LAB - SIMULATION OVERVIEW

B1.1 Business Rationale and Problem Statement

Despite the Indian mutual fund industry being among the world's fastest-growing, training in directional, calibrated, and audit-defensible regime detection remains virtually inaccessible to early-career analysts. Buy-side desks at Tier 1 fund houses recruit analysts who can read macro context, parse breadth and flow signals, and form probabilistic views about market direction - yet no platform teaches this integrated workflow with the rigour modern Bayesian methods demand.

Regime Lab fills this gap by providing a gamified environment where trainees build, calibrate, ensemble, and compete with AI-augmented regime detection systems across realistic Indian market scenarios spanning 2007 to the present.

The training gap this platform solves:

- Indian finance curricula teach portfolio theory but not regime-aware allocation methodology
- Existing CFA / NISM programmes provide risk-classification concepts but no calibration training
- No platform teaches integration of Bayesian deep learning, foundation models, regime-switching VAR, sequential inference, and conformal prediction into a single ensembled engine
- Sequential/online model maintenance for buy-side investment desks is untaught in formal education

Target audience:

- Tier 1: MBA/MSF early career talent at IIMs, ISB, FMS with quantitative finance specialisation
- Tier 2: Engineering early career talent at IITs/IISc/NITs interested in buy-side roles
- Tier 3: Early-career analysts at AMCs, mutual funds, and family offices
- Tier 4: Independent financial advisors and registered investment advisors (RIAs) supplementing their decision frameworks

B1.2 Simulation Architecture

Component	Description	Technology

Component	Description	Technology
Regime Engine	Generates regime probabilities from HMM, RS-VAR, BNN, foundation-head, and ensemble sources	Python, PyMC, NumPyro, TensorFlow, hmmlearn
Online Inference	Particle filter + BOCPD for intraday state tracking between batch refits	Python, NumPy, filterpy
Market Replay	Replays Indian historical data 2007-present with configurable speed	Python, FastAPI, Redis, PostgreSQL
Backtesting Engine	Runs regime-overlay allocation, Information Ratio, regime-conditioned Monte Carlo	Python, vectorbt / custom
Scoring Engine	Evaluates regime calibration, ensembling quality, allocation quality, scheme P&L	Python, PostgreSQL, conformal package
Leaderboard	Competitive ranking across cohorts and institutions	React, Node.js, MongoDB
Scenario Generator	Injects RBI events, election shocks, FPI sudden stops	Python, regime-conditioned MC
Audit Module	Generates Investment Committee artefacts for every player decision	Python, Jinja2 templating, PDF generation

B1.3 Game Modes

Campaign Mode (Primary - 9 Levels)

- Level 1 - The Indicator Reader: Build a regime classifier using only Nifty trend, VIX, and breadth. Achieve >55% accuracy across 2015-2018 test window.
- Level 2 - The Macro Strategist: Add macro features (rates, INR, FII flows). Improve regime accuracy by 10% over Level 1.
- Level 3 - The HMM Builder: Fit a Gaussian-HMM to Indian returns. Identify the 2008, 2013, 2018, 2020 regime breaks within 5 trading days.
- Level 4 - The Bayesian Sceptic: Replace point HMM with a Bayesian HMM via PyMC. Deliver calibrated regime probabilities with conformal coverage $\geq 88\%$.
- Level 5 - The Multivariate Modeller: Build a regime-switching VAR over returns, vol, breadth, and flows. Recover regime-conditional cross-asset covariance with credible intervals.
- Level 6 - The Deep Learner: Train a Bayesian neural network for regime classification. Beat HMM on rolling out-of-sample log-likelihood.

- Level 7 - The Foundation Engineer: Fine-tune at least two time-series foundation models (Chronos / TimesFM / Lag-Llama) for the Indian regime task. Demonstrate sample efficiency vs. trained-from-scratch baseline.
- Level 8 - The Ensembler: Combine HMM, RS-VAR, BNN, and foundation-head sources via stacking / Bayesian model averaging. Beat every individual member on out-of-sample calibrated log-likelihood.
- Level 9 - The Regulator's Choice: Survive a simulated SEBI inspection - every regime call must have full lineage, calibration metric, online/batch reconciliation, and Risk-O-Meter consistency. Audit-defensibility is the binding constraint.

Stress Test Mode

Players are dropped into historical crisis episodes (2008 GFC, 2013 Taper Tantrum, 2018 IL&FS, 2020 COVID, 2024 election results, hypothetical 2026 scenarios) with hidden ground-truth regimes. They must call regime probabilities in real time and explain their reasoning. Scoring weights calibration, speed, and ex-post Investment Committee acceptance.

Coach Mode

A senior CIO persona provides feedback on every regime call - calling out missed signals, miscalibrated confidence, and inconsistent allocation logic. Coach Mode is designed for use by AMC training programmes and B-school cohorts under instructor guidance.

SECTION B2: SCORING SYSTEM AND GAME MECHANICS

B2.1 1000-Point Scoring Framework

Category	Points	Criteria
Regime Accuracy	200	Posterior probability assignment vs. labelled historical regimes; log-likelihood scoring
Calibration Quality	200	Conformal coverage achieved (incl. online/time-series methods), Expected Calibration Error (ECE), reliability diagram smoothness
Bayesian Rigour	200	Use of priors, posterior credible intervals, MCMC diagnostics (R-hat, ESS, divergences), sequential inference correctness
Model Architecture & Ensembling	150	Breadth of complementary models (HMM, RS-VAR, BNN, foundation head), quality of stacking / BMA combination, model-selection evidence (WAIC/LOO)
Allocation Logic	100	Coherence of allocation tilts with regime calls; Information Ratio of regime-driven overlay
Audit Defensibility	100	Full feature lineage, model versioning, online/batch

Category	Points	Criteria
		reconciliation, decision-rationale documentation
Speed & Reporting	50	Time to regime call, presentation clarity, written documentation quality

B2.2 Progression System (9 Levels)

Level	Title	Points Required	Unlock
1	Indicator Novice	0	Basic trend, VIX, breadth tools
2	Macro Apprentice	120	Macro feature library and flow signals
3	HMM Engineer	240	Frequentist HMM library with hmmlearn
4	Bayesian Sceptic	360	PyMC NUTS sampling for HMM posteriors
5	Multivariate Modeller	480	Regime-switching VAR, multivariate Bayesian dynamics
6	Deep Regime Architect	600	Bayesian neural networks, MC dropout, ensembles
7	Foundation Practitioner	720	Chronos, TimesFM, Lag-Llama, Moirai checkpoints
8	Ensemble Strategist	850	Stacking, BMA, particle filter, BOCPD online inference
9	Regulator's Choice	950	Full audit-trail simulation, SEBI inspection scenarios

B2.3 Sample Scenario: The 2024 Election Verdict Surprise

Scenario: On 4 June 2024, the ruling alliance's vote share comes in below pre-poll forecasts. Nifty 50 gaps down ~6% at open before recovering through the session to close down ~5.9%. India VIX spikes from 14 to 27. FII selling intensifies.

- Pre-event regime state: Risk-On with high conviction; risk-on probability ~70%, conviction ~75%
- Player must: Recognise that an election outcome event is a discontinuity not captured by smooth regime models; widen prediction intervals; defer high-conviction tilts until post-event regime estimate stabilises
- Expected behaviour: Reduce beta exposure modestly pre-event; let conformal prediction set widen; let the BOCPD changepoint detector and regime engine reclassify within 5 trading days based on post-event indicators

- Scoring: 50 points for pre-event uncertainty widening, 30 points for post-event re-calibration speed, 20 points for not panic-selling

B2.4 Additional Sample Scenarios

Scenario 2: The 2020 COVID Crash

Nifty falls ~40% in March 2020. India VIX peaks above 80. SIP inflows hold up; FII selling persistent. Within 12 months, Nifty recovers to all-time highs. The regime engine must distinguish acute Risk-Off from regime-Late-Cycle exhaustion.

- Player must: Switch from Risk-On (Feb 2020) to Risk-Off (mid-March) within 10 trading days
- Then identify Post-Shock transition (April-May 2020) as drawdown stabilises
- Then call Risk-On re-entry (June-Aug 2020) as breadth rebuilds
- Scoring: 40 points for crash detection speed, 30 points for Post-Shock identification, 30 points for early Risk-On re-entry

Scenario 3: The 2018 IL&FS Credit Shock

IL&FS defaults trigger a credit-market freeze in September 2018. Mid-cap and small-cap indices crash 25-35%. Large-cap Nifty falls only ~14%. The regime engine must detect the sector-specific stress hidden under benign large-cap behaviour.

- Player must: Use breadth (% above 50-DMA), small-cap relative performance, and credit-spread features to call Late-Cycle / Risk-Off well before the Nifty signal arrives
- Scoring: 50 points for early small-cap stress detection, 30 points for breadth feature weighting, 20 points for credit-spread integration

Scenario 4: The 2013 Taper Tantrum

US Fed signals tapering in May 2013. INR depreciates from 55 to 68 against USD over four months. FII outflows accelerate. Nifty correction is moderate (~10%) but EM-specific stress is acute. The regime engine must distinguish EM-driven stress from broad equity risk-off.

- Player must: Use INR depreciation, FII outflow, and Gilt yield features rather than Nifty-only signal
- Scoring: 40 points for EM-stress feature emphasis, 30 points for INR depreciation integration, 30 points for managed tilt rather than panic exit

Scenario 5: The 2017 Demonetisation Rally

November 2016 demonetisation initially triggers a sharp Nifty drop. By March 2017, Nifty has rallied to new highs. The regime engine must avoid the trap of confusing acute-shock noise with regime change.

- Player must: Recognise Post-Shock transition within 8 weeks; not call sustained Risk-Off

- Use breadth recovery and DII inflow as early Post-Shock signals
- Scoring: 40 points for Post-Shock timing, 30 points for DII inflow weighting, 30 points for avoiding sustained-bearish call

PART C: CASE STUDIES AND REAL-WORLD APPLICATIONS

SECTION C1: THE 2020 COVID REGIME TRANSITION

C1.1 Background

In late February and through March 2020, the COVID-19 pandemic triggered the fastest equity market drawdown in modern Indian history. Nifty 50 fell from 12,201 (14 January 2020) to 7,610 (23 March 2020) - a drawdown of 37.6% in nine weeks. India VIX spiked from ~13 to a peak of 86.6 on 24 March 2020. Foreign portfolio investors recorded their largest-ever single-month outflow (-₹61,973 crore in March 2020). By the end of 2020, however, Nifty had recovered to 13,981 - a new all-time high.

C1.2 Regime System Analysis

Pre-shock regime state: Most production regime models in early February 2020 were classifying the market as Risk-On with high conviction. The signals were genuinely benign: Nifty above 200-DMA, VIX below 14, FII flows positive, breadth above 65%. There was no precedent in 30+ years of Indian equity data for the velocity of the regime transition that followed.

What the regime engine should have detected: Sharply rising implied volatility from late February (VIX above 25 by 28 February), breadth collapse (% above 50-DMA below 30% by 6 March), FII outflows acceleration (-₹17,000 crore in first week of March), credit spreads widening, USDINR depreciation. A conformal-equipped Bayesian model with proper macro features - and a particle-filter / BOCPD online layer - would have widened prediction intervals and shifted probability mass toward Risk-Off within the first week of March.

C1.3 Quantitative Analysis

Hypothetical regime-driven allocation overlay performance:

Allocation Strategy	Drawdown (Mar 2020)	End-2020 Return	Notes
100% Equity (Passive)	-37.6%	+14.9%	Buy-hold benchmark
Static 80/20 Eq/Cash	-30.1%	+12.3%	Standard tactical strategy
Regime Overlay (Reactive)	-22.4%	+18.7%	Trim beta on regime transition signal
Regime Overlay (Calibrated)	-19.1%	+21.2%	Conformal-aware widening of intervals

C1.4 Python Detector for COVID-Style Regime Stress

The crisis-detection logic is implemented here as a model-side composite signal that feeds the engine directly. It counts how many independent stress channels fire on a given day:

```
import numpy as np
```

```
import pandas as pd

def covid_style_crash_alert(feat: pd.DataFrame) -> pd.Series:
    """Composite crisis detector. feat must contain pre-computed columns:
       vix_z (252d z-score), pct_above_50dma, fii_flow_z_60d, usdinr_z_60d."""
    vix_spike       = feat['vix_z'] > 2.5
    breadth_collapse = feat['pct_above_50dma'] < 25
    fii_outflow     = feat['fii_flow_z_60d'] < -1.5
    inr_stress      = feat['usdinr_z_60d'] > 1.5
    signal_count = (vix_spike.astype(int) + breadth_collapse.astype(int)
                    + fii_outflow.astype(int) + inr_stress.astype(int))
    return pd.cut(signal_count, bins=[-1, 0, 1, 2, 4],
                  labels=['NORMAL', 'MONITOR', 'WARNING', 'ACUTE RISK-OFF'])

# Feed the count (or a soft version) as an additional regime feature, and use
# 'ACUTE RISK-OFF' days as high-weight calibration anchors for the Risk-Off state.
```

SECTION C2: THE 2018 IL&FS CREDIT SHOCK

C2.1 Background

On 14 September 2018, IL&FS Financial Services defaulted on its short-term commercial paper, triggering the worst Indian non-bank credit shock since 2008. The Nifty 50 large-cap index fell ~14% from its August 2018 peak by October - a moderate drawdown. The Nifty Midcap 100, however, fell ~25% over the same period; the Nifty Smallcap 100 fell ~35%. Credit spreads on AAA bonds widened to multi-year highs.

C2.2 Regime Implications

A regime engine relying on Nifty alone would have called this a Late-Cycle or mild Transitional regime; the truth on the ground was acute Risk-Off in mid- and small-caps, decoupled from large-cap behaviour. The lesson: regime engines must include breadth, credit-spread, and capitalisation-segmented features - not only headline index returns. This is precisely the kind of cross-asset, cross-cap structure the regime-switching VAR of Section A8 is designed to capture.

C2.3 Python Cap-Segmented Stress Signal

The capitalisation-divergence detector - flagging when small-cap conviction collapses while large-cap conviction holds - is implemented as a model-side feature:

```
def cap_segmented_stress(conviction_by_cap: dict, threshold: float = 0.4) -> str:
    """conviction_by_cap: {'Large Cap': float, 'Mid Cap': float, 'Small Cap': float}
       where each value is the regime-conviction score for that segment."""
    divergence = conviction_by_cap['Large Cap'] - conviction_by_cap['Small Cap']
    return 'CAP_DIVERGENCE_WARNING' if divergence > threshold else 'ALIGNED'

# A persistent positive divergence (large-cap calm, small-cap stressed) is the
```

```
# IL&FS signature; route it into the Risk-Off emission likelihood for mid/small.
```

SECTION C3: THE 2013 TAPER TANTRUM

C3.1 Background

In May-September 2013, US Federal Reserve Chair Ben Bernanke signalled the eventual tapering of QE. EM currencies depreciated sharply; the INR fell from approximately 55 to 68 against the USD between May and August 2013. FII outflows from Indian equities accelerated. Nifty correction was moderate (~10%) but EM-specific stress was acute. The RBI eventually responded with rate hikes and special FCNR(B) windows.

C3.2 Regime Implications

A regime engine purely tied to Nifty would have called this a moderate Late-Cycle correction. The actual structural stress was EM-specific - visible in INR, FII flows, and the credit-spread term structure but not in headline Indian equity prices. The lesson: regime engines must include cross-asset features (currency, rates, FII flows) to detect EM-specific stress regimes that hide in equity index data - again, a direct argument for the multivariate RS-VAR.

SECTION C4: THE 2017-2018 MID-CAP RALLY AND CORRECTION

C4.1 Background

Through 2017, the Nifty Midcap 100 rallied ~47%; the Nifty Smallcap 100 rallied ~58%. SEBI's October 2017 categorisation rules forced fund houses to re-bucket existing schemes. As large-cap demand from re-bucketed multi-cap schemes evaporated and mid/small-cap valuations stretched, the segment corrected ~25% through 2018. This was a classic Late-Cycle regime that was masked by continued large-cap strength.

C4.2 Regime Implications

A multi-feature regime engine combining valuation z-scores (mid-cap PE relative to long-run mean), breadth indicators (mid-cap participation), and flow indicators (HNI flows into mid-cap-heavy schemes) would have identified Late-Cycle by Q3 2017. The point forecast: “Nifty Midcap will fall 25%” was untestable in real-time and indefensible. The probability statement: “P(Late-Cycle | features) > 0.6 by Q3 2017” was testable, defensible, and actionable.

> *Direction over price. A point forecast that a mid-cap correction is coming is a guess. A calibrated probability that a Late-Cycle regime is in force is an institutional view.*

SECTION C5: THE 2024 ELECTION VERDICT EVENT

C5.1 Background

On 4 June 2024, the ruling alliance's seat count came in below pre-poll forecasts. Nifty gapped down ~6% at open and ultimately closed down ~5.9% on the day. India VIX spiked from 14 to 27 intraday. By 7 June, with formation of a coalition government clarified, Nifty had recovered

most of the loss. The episode was an acute event-driven shock embedded in a stable Risk-On regime.

C5.2 Regime Implications

Event-driven shocks differ fundamentally from regime transitions. A well-designed regime engine must distinguish between (a) genuine regime shift requiring allocation re-anchoring, and (b) event-driven volatility that resolves within days and does not warrant strategic action. The conformal prediction wrapper plays a critical role here: pre-event, the prediction interval widens to acknowledge event uncertainty; post-event, the interval re-narrows as the resolution becomes clear.

C5.3 Python Calendar-Aware Regime Adjustment

The known-event calendar dampens model conviction around scheduled discontinuities - implemented as a model-side post-processing step on the ensembled conviction score:

```
def event_adjusted_conviction(base_conviction: float,
                              days_to_event: int) -> float:
    """Halve conviction within 5 days of a known scheduled event
       (election count day, RBI policy, budget, major macro print)."""
    near_event = 0 <= days_to_event < 5
    return base_conviction * 0.5 if near_event else base_conviction

# Apply AFTER ensembling, BEFORE the tilt-recommendation rule, so the engine
# defers high-conviction tilts across known discontinuities.
```

PART D: 15-DAY PROJECT METHODOLOGY AND DELIVERABLES

SECTION D1: OVERVIEW

This project follows an AI-accelerated agile methodology where early career talent leverage LLM tools to compress what would traditionally be a three-month research project into 15 intensive days. The 80/20 rule applies: 80% AI-generated code with 20% custom domain integration, calibration, and Investment Committee-grade validation. The methodological emphasis is on model depth, calibration discipline, and audit-defensibility - building multiple complementary regime models and combining them rigorously - which is the explicit antithesis of price-prediction quant practice.

The project submission will be in the form of a GitHub Repository Ownership Transfer (you can click the icon on task 2 of the project to transfer the repository) to the ZethetaIntern user id.

SECTION D2: DAY-BY-DAY BREAKDOWN

Day 1 - Domain Orientation and Environment Setup

- Read this entire training guide (Parts A through F) thoroughly, with emphasis on Sections A1, A2, A8, A9 and A10
- Set up Python environment: conda create -n regime python=3.10
- Install required libraries: numpy, pandas, scipy, scikit-learn, hmmlearn, statsmodels, pymc, numpyro, arviz, tensorflow, tensorflow-probability, torch, torch_geometric, chronos-forecasting, timesfm, gtda, shap, mapie, crepes, filterpy, ruptures
- Download sample Indian market data: Nifty 50, Nifty Midcap 100, Nifty Smallcap 100, India VIX, USD/INR, 10Y Gilt, FII/DII flows (provided CSV pack or Yahoo Finance + manual macro)
- Set up Git repository (PRIVATE) and initialise project structure
- Deliverable: Environment confirmation screenshot, initial project README

Day 2 - Indian Market Data Ingestion and Exploration

- Load OHLC data for Nifty 50, Nifty Midcap 100, Nifty Smallcap 100 (15-year history minimum)
- Load India VIX, USD/INR, 10Y Gilt, AAA-Gilt spread, FII/DII daily flows, monthly SIP totals
- Compute basic statistics: mean, std, skewness, kurtosis of returns by capitalisation segment
- Visualise: price paths, return distributions, VIX paths, FII/DII flow series
- Implement data quality checks: missing values, weekend gaps, corporate actions
- Calculate rolling correlations between large/mid/small and across asset classes
- Deliverable: Jupyter notebook with exploratory analysis, data quality report

Day 3 - Feature Engineering for Regime Classification

- Implement the feature engineering pipeline from Section A4.5 (30+ features)
- Add cap-segmented features (mid-cap vs large-cap relative performance, mid-cap valuation z-score)
- Add flow features (FII/DII z-scores, SIP momentum, flow balance ratio)
- Add macro features (RBI policy stance proxy, real rates, INR DXY-orthogonalised)
- Add topological features (persistence landscapes of rolling correlation matrices, Section A7.2) and sector-GCN embeddings (Section A7.3)
- Visualise feature time series with overlaid major regime breakpoints (2008, 2013, 2020, 2024)
- Deliverable: Feature engineering module (including TDA + GNN features), feature visualisation notebook

Day 4 - Frequentist HMM Implementation

- Fit a 5-state Gaussian HMM to Nifty returns using hmmlearn
- Implement post-hoc regime labelling (Risk-On, Late-Cycle, Transitional, Post-Shock, Risk-Off)
- Generate regime sequences and probability paths; visualise regime overlays on the Nifty path
- Compute regime durations, transition matrix, and stickiness statistics
- Compare 3-state, 5-state, 7-state HMMs by BIC
- Deliverable: HMM training notebook, regime visualisations, transition matrix

Day 5 - Bayesian HMM via PyMC

- Implement the Bayesian HMM from Section A3.4 with Dirichlet priors on transitions
- Run NUTS sampler (2000 draws, 1000 tune, 4 chains)
- Check MCMC diagnostics: R-hat, effective sample size, posterior trace plots, divergences
- Generate posterior credible intervals on transition probabilities and regime durations
- Compare Bayesian vs frequentist regime probability paths visually
- Deliverable: Bayesian HMM model, diagnostic report, comparison notebook

Day 6 - Regime-Switching VAR and Multivariate Dynamics

- Fit the single-feature Markov-switching baseline (statsmodels, Section A8.2)
- Build the full multivariate Bayesian RS-VAR from Section A8.3 (PyMC or NumPyro) over returns, vol, breadth, FII/DII, INR, gilt
- Recover regime-conditional VAR coefficients and innovation covariance with credible intervals

- Generate regime-conditional impulse responses (e.g. FII-outflow shock propagation by regime)
- Validate MCMC diagnostics; compare smoothed regime paths against the Bayesian HMM
- Deliverable: RS-VAR model, impulse-response notebook, regime-conditional covariance report

Day 7 - Bayesian Deep Learning

- Build the MC Dropout regime classifier from Section A4.2
- Build the variational Bayesian neural network from Section A4.3
- Build the deep ensemble from Section A4.4 (M=10 models)
- Compare all three on time-series cross-validation
- Decompose uncertainty into epistemic and aleatoric components; generate SHAP attributions for selected dates
- Deliverable: BDL training scripts, uncertainty visualisations, SHAP attribution plots

Day 8 - Foundation Model Integration (Multiple Models)

- Install and test the Chronos pipeline; generate embeddings on rolling 252-day windows
- Integrate at least one further foundation model (TimesFM, Lag-Llama, or Moirai) and generate comparable embeddings/quantiles
- Train a Bayesian classification head on top of each foundation embedding
- Compare hybrid (foundation + Bayesian head) vs. trained-from-scratch BDL on sample-efficiency curves, across the two foundation models
- Deliverable: Multi-foundation hybrid implementation, sample-efficiency comparison report

Day 9 - Model Ensembling, Stacking, and Selection

- Assemble out-of-fold regime probabilities from HMM, RS-VAR, BNN, and foundation-head members on a strict time-respecting split
- Implement Bayesian Model Averaging (Section A10.1) and constrained stacking (Section A10.2)
- Run WAIC / PSIS-LOO model comparison in ArviZ for the Bayesian members (Section A10.3)
- Show the ensemble beats every individual member on out-of-sample calibrated log-likelihood
- Define and implement the combined regime output contract (Section A10.4)
- Deliverable: Ensembling module, model-comparison report, combined-output schema

Day 10 - Sequential and Online Regime Inference

- Implement the bootstrap particle filter for online regime state (Section A9.1)

- Implement Bayesian Online Changepoint Detection (Section A9.2) and validate firing on 2018/2020 breaks but not on routine noise
- Implement streaming Dirichlet/Beta posterior updates and the nightly-batch / intraday-online two-speed design
- Build the online/batch reconciliation diagnostic (Section A9.3)
- Deliverable: Sequential-inference module, changepoint validation notebook, reconciliation report

Day 11 - Conformal Prediction and Calibration

- Implement split-conformal classifier (A6.2) and Adaptive Prediction Sets (A6.3)
- Implement at least one distribution-shift-robust method: EnbPI, Adaptive Conformal Inference, or Mondrian conformal (A6.5)
- Generate reliability diagrams and ECE for all regime models and for the ensemble
- Run on rolling 252-day windows to track coverage stability; compare base vs conformalised vs online-conformal coverage
- Deliverable: Calibration module, reliability diagrams, conformal coverage report

Day 12 - Indian Market Case Study Research and Replication

- Research and document four Indian case studies (Section C of this guide as starting point)
- Quantify P&L impact of each event on illustrative model portfolios
- Analyse how the trained ensemble engine would have performed in each episode, including BOCPD changepoint timing
- Propose regime-specific adaptations (feature additions, prior modifications, event-day handling)
- Deliverable: Case study report (minimum 8 pages, with charts and quantitative tables)

Day 13 - Backtesting, Monte Carlo, and Allocation Overlay

- Build the regime-conditioned Monte Carlo engine from Section A13 (path simulation, VaR/CVaR)
- Connect ensembled regime probabilities to a scheme allocation overlay (tilt rules conditioned on regime + conviction)
- Backtest the overlay over 2019-2024; compute Information Ratio, tracking error, and regime-conditioned drawdowns vs buy-hold
- Generate the Investment Committee artefact: conditional statements (Section A13.3) with full lineage
- Deliverable: Backtesting & simulation engine, IR/overlay results, IC artefact template

Day 14 - Validation, Polish, and Integration

- End-to-end validation of the regime pipeline: raw data ingestion through ensembled regime probability to tilt recommendation to Investment Committee artefact
- Cross-validate metrics between Python and R implementations (HMM probabilities, calibration, VaR)
- Polish all visualisations; verify the online-inference loop runs end-to-end on a streamed day
- Write final report sections, executive summary, and Investment Committee briefing template
- Peer review of code and documentation
- Deliverable: Validated, polished, integrated deliverables

Day 15 - Final Submission and Handover

- Transfer GitHub repository ownership to @ZethetaIntern
- Upload all deliverables to shared drive
- Submit final report, code, and presentation
- Record 10-minute demonstration video walking through a live regime call from raw data to Investment Committee artefact
- Complete exit documentation and knowledge transfer
- Deliverable: All six deliverables submitted and handed over

SECTION D3: MANDATORY DELIVERABLES CHECKLIST

Deliverable 1: Main Report

- Minimum 40 pages covering theory, methodology, results, case studies, calibration evidence, and the ensembling design
- All formulas must be shown with derivations
- Regime model comparison tables with statistical significance, WAIC/LOO, and calibration tests
- Backtest results with regime-overlay allocation, Information Ratio, and Investment Committee narrative

Deliverable 2: Python Codebase

- Feature engineering module (returns, trend, vol, breadth, flow, macro, TDA, GNN features)
- Frequentist HMM and Bayesian HMM (PyMC) implementations
- Multivariate regime-switching VAR (PyMC / NumPyro)
- Bayesian deep learning models (MC Dropout, variational, ensemble)
- Foundation model integration (Chronos plus at least one of TimesFM / Lag-Llama / Moirai)

- Sequential inference: particle filter and Bayesian online changepoint detection
- Model ensembling layer: Bayesian model averaging and constrained stacking, with WAIC/LOO selection
- Conformal prediction wrapper (split, APS, and a distribution-shift-robust variant)
- Monte Carlo regime-conditioned simulation engine

Deliverable 3: R Codebase

- HMM and Markov-switching regression in R (depmixS4, MSwM)
- Bayesian regime model in Stan or rstanarm; changepoint detection (bcp / changepoint)
- Conformal prediction in R (conformal or mlr3 packages)
- Cross-validation script reconciling R regime probabilities against the Python engine

Deliverable 4: Backtesting and Simulation Engine

- Regime-overlay allocation backtester (2019-2024) with Information Ratio and tracking error
- Regime-conditioned Monte Carlo path/VaR engine
- Scenario replay harness for the Part C crisis episodes
- Investment Committee artefact generator (conditional statements with full lineage)

Deliverable 5: Model Card, Calibration and Validation Pack

- Model card documenting every member model, its inputs, priors, and version
- Reliability diagrams, ECE, and rolling conformal coverage for each model and the ensemble
- MCMC diagnostics (R-hat, ESS, divergences) for all Bayesian models
- Online/batch reconciliation evidence and cross-language (Python vs R) numerical checks

Deliverable 6: Final Presentation

- 18-slide presentation covering methodology, results, calibration evidence, ensembling design, and gamification concept
- 10-minute recorded demonstration video walking through a live regime call from raw data to Investment Committee artefact

PART E: TOOLS AND PLATFORMS GUIDE

SECTION E1: PYTHON ENVIRONMENT

E1.1 Required Libraries

```
pip install numpy pandas scipy scikit-learn matplotlib seaborn plotly
pip install hmmlearn statsmodels pymc numpyro arviz
pip install tensorflow tensorflow-probability torch torch_geometric
pip install chronos-forecasting timesfm
pip install gtda shap mapie crepes
pip install filterpy ruptures bayesian-changepoint-detection
pip install yfinance nsetools beautifulsoup4 requests
```

Essential Python packages and their roles:

Package	Purpose	Version
pandas	Data manipulation and time series	>=2.1
numpy	Numerical computing	>=1.26
scikit-learn	Cross-validation, calibration, baseline ML	>=1.4
hmmlearn	Frequentist Hidden Markov Models	>=0.3
statsmodels	Markov-switching regression / RS-VAR baseline	>=0.14
pymc / numpyro	Bayesian HMM, RS-VAR, probabilistic programming	>=5.10 / >=0.13
arviz	MCMC diagnostics, WAIC/LOO, posterior visualisation	>=0.17
tensorflow / tensorflow-probability	Bayesian neural networks	>=2.15 / >=0.23
chronos-forecasting / timesfm	Pre-trained time-series foundation models	latest
mapie / crepes	Conformal prediction wrappers	>=0.8
filterpy / ruptures	Particle filtering and changepoint detection	>=1.4 / >=1.1
gtda	Topological data analysis	>=0.6
shap	Feature attribution explainability	>=0.44

SECTION E2: R ENVIRONMENT

```
install.packages(c(
  "tidyverse", "data.table", "lubridate",
  "depmixS4", "MSwM", "rstanarm", "brms",
  "bcp", "changepoint", "bsts",
```

```
  "caret", "xgboost", "randomForest",
  "quantmod", "TTR", "PerformanceAnalytics",
  "reticulate", "conformalInference"
))
```

SECTION E3: BACKTESTING AND EXPERIMENT TRACKING

Because the deliverables centre on model building rather than reporting surfaces, the supporting toolchain emphasises reproducible backtesting and experiment tracking:

- vectorbt / custom backtester: fast vectorised backtesting of the regime-overlay allocation, Information Ratio, and tracking error against a buy-hold benchmark
- MLflow (or Weights & Biases): track every model run - hyperparameters, priors, MCMC diagnostics, calibration metrics, and ensemble weights - so results are reproducible and comparable across the candidate's iterations
- DVC (optional): version the Indian market data snapshots used for each backtest so results are exactly reproducible
- ArviZ: standardised model-comparison reports (WAIC, PSIS-LOO) and posterior diagnostics for the Bayesian members of the ensemble

Every model produced for this project must be logged to the experiment tracker with its calibration and diagnostic metrics before it is admitted to the ensemble.

SECTION E4: INDIAN MARKET DATA SOURCES

Source	Data	Access	Cost
NSE Indices	Nifty family, sector indices	Web scrape / NSE API	Free
NSE Bhavcopy	OHLC for all listed stocks	Daily CSV download	Free
RBI Database (DBIE)	Repo, CRR, SLR, FX, money supply, gilt yields	Web download	Free
MOSPI	CPI, IIP, GDP	Web download	Free
SEBI	Mutual fund AUM, scheme disclosures	Web download	Free
AMFI	MF flow statistics, SIP totals	Monthly Excel files	Free
Yahoo Finance (yfinance)	Daily OHLC, FX, indices	Python API	Free
Tijori / Sensibull / Smallcase	Derived analytics, F&O data	REST API	Subscription
Bloomberg / Refinitiv	Institutional-grade tick and macro data	Terminal / API	Paid

SECTION E5: AI RESEARCH TOOLS

Permitted AI tools and usage guidelines:

- Claude.ai: Primary code generation, documentation, and architecture-review assistant
- ChatGPT / GPT-4: Secondary code generation and debugging
- Cursor: AI-integrated IDE for code completion and refactoring
- GitHub Copilot: In-editor code suggestions

Mandatory validation requirements:

- ALL AI-generated code must be tested with real Indian market data before submission
- ALL Bayesian models must include posterior diagnostic checks (R-hat, ESS, divergences)
- ALL ensemble members must be logged to the experiment tracker with calibration metrics before admission to the ensemble
- ALL regime probabilities must be conformalised and have empirical coverage documented (including under distribution shift)
- ALL financial calculations must be cross-checked between the Python engine and the R codebase

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research and Analysis, Presentation and Clarity, Innovation and Creativity, Feasibility and Practicality, and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

Submission must include all six deliverables enumerated in Part D, Section D3. The GitHub repository must be transferred to ZethetaIntern at the time of submission, and read access to the deliverables shared drive must be granted to the Zetheta assessment team. All written deliverables must follow the standard Zetheta template, retain the CIN (U62012MH2023PTC410415) in document footers, and be supplied in both PDF and editable formats.

Assessment outputs from Zetheta are indicative and will be shared with the engaging party (other potential employers or the relevant institution), who retains sole discretion over any downstream decision regarding internships, employment, awards, or further engagement.

END OF PROJECT DOCUMENT